



MAY UNIVERSITY IN CAIRO

FACULTY OF ENGINEERING

Computer Engineering Program

Swarm Emergency Robotics System

Autonomous Multi-Agent Coordination for Disaster Response

Graduation Project

Spring 2026

Prepared By:

Marwan Ramadan Ahmed
Mahmoud Mohamed Matouk
Mahmoud Naser Abdelkareem
Mohamed Ahmed Tawfik
Yehia Mostafa Gaber

Abdulrahman Ahmed Fathy
Mohamed Esmael Mahmoud
Momen Mohamed Sharkawi
Hamed Ahmed Nabih

Supervised By:

Dr. Hassan Ibrahim

Committee Report

We certify we have read this graduation project report as examining committee, examine the student in its content and that in our opinion it is adequate as a project document for “
”.

Supervisors:

Name:

Signature:

Date: / /202

Examiners:

Name:

Signature:

Date: / /202

Intellectual Property Right Declaration

This is to declare that the work under the supervision of Dr. Hassan Ibrahim having title Swarm Emergency Robotics System carried out in partial fulfillment of the requirements of Bachelor of Engineering at Faculty of Engineering is the sole property of May University In Cairo and the respective supervisor. It is protected under the intellectual property right laws and conventions. It can only be considered/ used for purposes like extension for further enhancement, product development, adoption for commercial/organizational usage, etc. with the permission of the University and respective supervisor. This above statement applies to all students and faculty members.

Names:

Supervisor:

Acknowledgement

We would like to express our deepest gratitude to Dr. Hassan Ibrahim for his invaluable guidance, continuous support, and expert supervision throughout the development of this Swarm Robot System project. His technical insights and constructive feedback have been instrumental in shaping the direction and quality of this research.

We extend our sincere thanks to the Department of Computer Engineering at May University in Cairo for providing us with the necessary resources and support required for the successful completion of this project.

We also acknowledge the challenges we faced as Computer Engineering students learning new domains such as electronics, mechanical design, and robotics hardware. The support from our colleagues and the broader robotics community through online resources and documentation was invaluable.

Finally, we would like to thank our families for their encouragement and understanding throughout the duration of this demanding academic endeavor.

Abstract

One example of an autonomous collaborative robotics approach is the Swarm Robot System, used in environments such as search and rescue. This project provides a framework for coordinating multiple autonomous robots to work together on tasks. The system architecture is based on three dedicated cooperative robotic platforms. The Alpha robot uses SLAM with LiDAR, with the `slam_toolbox` to create accurate 2D maps of the environment. The Beta robot is equipped with the camera for YOLO based fire, smoke and flame detection and water-pump mechanism for fire intervention. Gamma is an ESP32-based gas sensing and environmental monitoring device that uses a MQ-5 sensor. An important element of our approach is the use of a common map representation to which all three robots contribute in a coordinated architecture. The software framework is built around the Robot Operating System 2 (ROS 2). Every Raspberry Pi based robot runs ROS 2 in a local installation. The Pi-based robots communicate with the operations center using ZeroMQ (ZMQ) with envelopes serialised using `msgpack` over a 5 GHz WiFi network, and the ESP32-based Gamma robot communicates using HTTP/JSON endpoints. As Computer Engineering students with limited exposure to electronics, mechanical design and robotics hardware, this project is also a major learning experience for the team. The team split responsibilities into programming, design, electronics and environment preparation, but kept cross-functional understanding. Now the system is capable of autonomous mapping and object detection in the mapped environment. The test environment is a 4 x 4 meter area consisting of four rooms.

Keywords: Swarm Robotics, Multi-Robot Systems, SLAM, Sensor Fusion, Object Detection, Search and Rescue, ROS, YOLO

Table of Contents

Acknowledgement	4
Abstract	5
Table of contents	6
List of figures.....	9
List of tables.....	10
List of abbreviations and acronyms.....	6
Chapter 1: Introduction	25
1.1 Project Overview	13
1.2 Background and Context	15
1.3 Problem Statement.....	17
1.4 Motivation.....	19
1.5 Project Objectives	20
1.6 Project Scope.....	22
1.7 General Constraints	23
1.8 Team Organization	24
1.9 Organization of the Dissertation.....	25
Chapter 2: Literature Review and Related Work.....	28
2.1 Introduction to Swarm and Cooperative Robotics	28
2.2 Review of Multi-Robot and Swarm Systems	28
2.2.1 Fundamental Concepts.....	29
2.2.2 Reviewed Projects and Systems.....	30
2.3 Review of Search and Rescue Robotic Systems.....	31
2.3.1 Existing SAR Robot Systems	32
2.3.2 Key Features Identified.....	32
2.4 Review of SLAM Techniques.....	33
2.4.1 SLAM Fundamentals	33
2.4.2 SLAM Algorithm Review.....	33
2.4.3 Sensor Technologies for SLAM	34
2.4.4 Sensor Fusion	35
2.5 Review of Object Detection in Robotics	36
2.5.1 Deep Learning for Object Detection	36
2.5.2 YOLO (You Only Look Once).....	36
2.5.3 Deployment Considerations	37
2.6 Review of Communication Frameworks.....	38
2.6.1 Robot Operating System (ROS)	38
2.6.2 Network Communication.....	38

2.6.3 Dashboard and Monitoring	39
2.7 Review of Hardware Components.....	40
2.7.1 Processing Platforms.....	40
2.7.2 Sensors.....	41
2.7.3 Motors and Drive Systems.....	41
2.7.4 Power Systems.....	42
2.8 Review of Mechanical Design Approaches	42
2.8.1 Chassis Design.....	42
2.8.2 Design Workflow	43
2.9 Summary of Literature Review.....	43
Chapter 3: System Analysis and Design.....	25
3.1 Project Planning and Feasibility	46
3.1.1 Development Methodology.....	46
3.1.2 Project Timeline and Gantt Chart	49
3.2 Problem Analysis and Need for the System.....	50
3.3 Requirements and Users.....	51
3.3.1 User and Stakeholder Description	51
3.3.2 Functional Requirements.....	52
3.3.3 Non-Functional Requirements	53
3.4 System Architecture	54
3.5 Hardware and Software Design	57
3.5.1 Hardware Design	57
3.5.2 Software Design	61
3.6 Communication and Dashboard Design.....	66
3.7 Risks, Assumptions, and Dependencies	70
Chapter 4: Implementation.....	74
4.1 Hardware Implementation.....	74
4.2 Software Implementation	78
4.2.1 SLAM Implementation.....	79
4.2.2 Object and Human Detection Implementation.....	7
4.2.3 Sensor Integration.....	82
4.2.4 Robot Communication Implementation.....	83
4.2.5 Dashboard Implementation.....	84
4.3 System Integration.....	84
Chapter 5: Testing, Results, and Evaluation.....	89
5.1 Testing Strategy and Environment	89
5.1.1 Testing Methodology	89

5.1.2 Physical Test Environment	90
5.1.3 Hardware and Network Setup	90
5.2 Test Cases and Metrics.....	91
5.2.1 SLAM and Mapping Metrics	91
5.2.2 Visual Perception Metrics	91
5.2.3 Sensor Fusion and Localization Metrics	92
5.2.4 Gas Detection and Network Communication Metrics	92
5.3 Experimental Results.....	93
5.3.1 SLAM and Mapping Results.....	93
5.3.2 Visual Perception Results	95
5.3.3 Sensor Fusion and Localization Results	96
5.3.4 Environmental Sensing Results.....	97
5.3.5 Network and Communication Results.....	99
5.4 Analysis and Discussion.....	100
5.4.1 System Strengths and Achievements.....	100
5.4.2 Limitations and Constraints	100
Chapter 6: Conclusion and Future Work.....	104
6.1 Conclusion.....	104
6.2 Achievements of the Project.....	104
6.3 Future Enhancements	106
6.4 Potential Real-World Deployment.....	107

List Of Figures

Figure 2.1 – Main hardware components.....	40
Figure 3.1 – Multi-robot system configuration	55
Figure 3.2 – Multi-Robot System Architecture and Communication Framework.....	56
Figure 3.3 – CAD model of the robot chassis.....	57
Figure 3.4 – Final Assembled Design of the Mapping Explorer Robot.....	58
Figure 3.5 – Software Stack and ROS Node Architecture.....	62
Figure 3.6 – Native PySide6 desktop monitoring dashboard.....	66
Figure 3.7 – Real-Time Dashboard Interface.....	69

List of tables

Table 1.1: Application domains of swarm robotic systems	16
Table 2.1: Classification of swarm robotics coordination strategies	28
Table 2.2: Comparison of multi-robot path planning approaches.....	30
Table 2.3: Summary of SLAM techniques used in mobile robots.....	34
Table 2.4: Comparison of object detection models used in robotic perception	37
Table 3.1: Main components and specifications of the Mapping Explorer Robot	59
Table 3.2: Specifications of sensors used in the robotic system	61
Table 3.3: Software technology stack used in the robotic system	65
Table 4.1: Power Architecture Components and Specifications	78
Table 5.1: SLAM and Mapping Configuration.....	95
Table 5.2: Object-Detection Pipeline Configuration and Throughput.....	96
Table 5.3: Localization and Odometry Calibration.....	97
Table 5.4: MQ-5 Gas Detection Parameters.....	98
Table 5.5: Network Communication Metrics.....	99

List OF Abbreviation

Acronym	Full Form
AI	Artificial Intelligence
CNN	Convolutional Neural Network
EKF	Extended Kalman Filter
GPS	Global Positioning System
IMU	Inertial Measurement Unit
LiDAR	Light Detection and Ranging
ML	Machine Learning
PWM	Pulse Width Modulation
ROS	Robot Operating System
SAR	Search and Rescue
SLAM	Simultaneous Localization and Mapping
TCP/IP	Transmission Control Protocol/Internet Protocol
WiFi	Wireless Fidelity
YOLO	You Only Look Once

Chapter 1

Chapter 1: Introduction

1.1 Project Overview

The Swarm Robot System project is an improvement in the design of multi-robot autonomous systems, in particular collaborative robotics for environmental exploration and hazard assessment. The aim of this project is to develop a framework for the coordination of multiple autonomous robots working together to perform tasks that would be difficult or dangerous for a single robotic unit.[1][2]

The main idea of this project is based on the principle of “Collaborative Intelligence” – distribution of computational and sensory tasks over several specialised robotic units, each of them performing specific functions while acting in coordination. the required functionality. To fulfil these needs, we use three specialised robots that perform all the functions together: environment mapping with SLAM technology, fire detection with deep learning, gas sensing for hazard monitoring, and fire intervention capabilities.[1], [3]

- **Project Context and Significance**

In modern applications such as search and rescue operations, disaster assessment, and structural inspection, the use of human personnel is often hazardous and time-consuming. The Swarm Robot System addresses these challenges by providing an autonomous platform that can systematically explore unknown environments, identify hazards, locate objects of interest, and provide actionable intelligence in real-time.

Our approach is novel in that it designs an integrated system where the capabilities of each robot are purposefully designed and the data from all robots contribute to a common, unified representation of the environment. The common map is the basis for decision making for all robots and allows them to behave in a coordinated manner.[4][1]

- **System Composition**

The system is made up of three autonomous mobile robots working together to carry out all required functions:

1. Alpha Robot (Mapping & Navigation) Equipped with RPLidar A1 LiDAR sensor and advanced on-board processing with Raspberry Pi 4 (8 GB RAM). This robot uses slam_toolbox for continuous SLAM and accurate 2D map generation of the environment, with rf2o laser odometry for motion estimation. Alpha has four JGB37-520

geared motors with Hall encoders, but these are not active at the moment; localisation is based on scan-based odometry (rf2o laser odometry feeding slam_toolbox).[4]

2. Beta Robot (Intervention and Fire Detection): Raspberry Pi 3B+, Logitech C270 USB camera, GY-87 IMU, 4x 25GA370 encoder motors (408 CPR), HC-SR04 ultrasonic sensors, water pump. This robot carries a fire-intervention water pump and hosts the camera for YOLO based fire detection. It uses custom complementary encoder and gyroscope odometry fusion for navigation and localisation.[5]
3. Gamma Robot (Environmental Monitoring) : It consists of an ESP32 DevKit micro-controller, MQ-5 gas sensor, HC-SR04 ultrasonic sensor, MPU6050 IMU and buzzer. This robot is designed to detect
4. monitoring of environmental conditions and toxic gases. Gamma uses HTTP/JSON endpoints and supports over-the-air (OTA) firmware updates. Gamma differs from the Pi-based robots in that it does not run ROS and operates independently with its own embedded HTTP interface.[5]

The robots communicate over a common WiFi network. The Raspberry Pi-based Alpha and Beta robots connect on 5 GHz, run ROS 2 locally, and expose telemetry, map data, video, and commands through a ZeroMQ/msgpack gateway protocol, while the ESP32-based Gamma robot has no 5 GHz radio and joins the network's 2.4 GHz band. The ESP32-based Gamma uses HTTP/JSON endpoints for telemetry and control. ROS 2 DDS traffic is limited to the local network of each robot on purpose, not broadcast over WiFi.[6]

• Technical Approach

The main technical features of this project are

- Shared Map Architecture: The Alpha robot uses LiDAR based SLAM with slam_toolbox to generate an occupancy grid map. The map is published on the ZMQ gateway and received on the PySide6 operations center that has a shared state store for all the connected robots.[5]
- Sensor Fusion. Alpha employs rf2o laser odometry in conjunction with slam_toolbox for localisation. Beta employs a custom complementary filter to fuse wheel encoder data with gyroscope readings, with a slip gate to reject unreliable measurements.
- Deep Learning Integration YOLO fire, smoke and flame detection runs in a crash-isolated child process on the operations center laptop with detections projected onto the shared map as incident markers.

- Real-time Dashboard: A native PySide6 desktop operations center with dockable panels for map visualisation, fleet status, live video with AI overlays, joystick / WASD teleop, fire and gas alerts, diagnostics and mission planning with A* route planning.[6]

1.2 Background and Context

• Project Background and Prior Research

A lot of research was done before starting the implementation of this project to understand the existing work in the field of multi-robot systems, SLAM technology and search and rescue robotics. This was a very important phase of the research since the members of the team are students of Computer Engineering and they had no prior experience in robotics hardware, electronics and mechanical design.[7]

In research phase, we studied many academic papers, open-source projects and existing implementations of similar systems. The principal subjects of study were:

- Previous implementations of mapping robots and SLAM algorithms
- Coordination frameworks and communication protocols for multiple robots
- Object detection systems using deep learning
- ROS (Robot Operating System) architecture and packages
- Sensor integration techniques for mobile robots

This review of previous work provided a foundation for what components would be needed, what challenges to expect, and what methods had worked in similar projects.[8]

• Evolution of Robotic Systems

The last 20 years have seen a complete transformation in the field of robotics. The earliest robotic systems were mostly fixed machines, dedicated to specific tasks in manufacturing environments. In mobile robotics new challenges were brought about: navigation in unstructured environments, real time decision making, management of sensor uncertainty and power constrained operation.[1]

The next step in robotics is an evolution to multi-robot systems. Instead of solving more and more complex problems with more and more complex individual robots, the research community has realised the benefits of distributed systems: redundancy, parallelisation of tasks, natural scalability and fault tolerance.

• Search and Rescue Applications

One of the most compelling use cases for autonomous robotic systems is search and rescue (SAR) operations. Traditional SAR operations are marked by:

- Environmental Hazards: Toxic atmosphere, extreme temperatures, unstable structures
- Information Scarcity – No previous maps or knowledge of the environment
- Time pressure: every hour of delay lowers the chances of survival
- Resource Limitations: shortage of trained personnel and specialised equipment

Robotic systems can help to overcome these problems by rapidly and methodically evaluating the environment, without endangering human personnel.[1], [4]

Table 1.1 gives a summary of the major application domains of swarm robotic systems, together with representative tasks and advantages.

Table 1.1: Application domains of swarm robotic systems

Application Domain	Example Tasks	Advantages
Search and rescue	Victim detection, mapping	Fast area coverage
Environmental monitoring	Pollution detection	Distributed sensing
Military	Surveillance	Robust coordination
Agriculture	Crop inspection	Scalability
Industrial automation	Warehouse logistics	Efficiency

• SLAM Technology

Simultaneous Localisation and Mapping (SLAM) has progressed from a theoretical concept to a practical and reliable technology. Key advances in SLAM include particle filter based methods (FastSLAM), visual SLAM systems (ORB-SLAM) and deep learning based SLAM. Our system is based on slam_toolbox, a modern and actively maintained SLAM framework in the ROS 2 ecosystem, together with rf2o laser odometry for scan-based motion estimation on the mapping robot.[5][9]

• Deep Learning for Object Detection

Deep learning has revolutionised how robots perceive their environment in robotic systems. Systems like YOLO for object detection are capable of detecting visual hazards with

high accuracy for robots. In our project, YOLO is mainly used for detecting fire, smoke and flames in the mapped environment, and the detection system supports selectable models through the PySide6 operations center.[10][11]

• **Robot Operating System (ROS)**

ROS has become the de facto platform for robotics research and development. ROS offers:

- Standardised Middleware Publish/Subscribe message passing Service calls
- Sensor Libraries: Dedicated driver support for a wide range of sensors.
- Algorithm Libraries Implementations of SLAM, motion planning and control algorithms.
- Simulation Environment ROS has integration with simulation tools like Gazebo. (This project does not use Gazebo; the operations center's own built-in --sim mode, which spawns simulated robots speaking the production protocol, provides hardware-free testing.)
- Community & Ecosystem: Well documented and prebuilt components.[6]

• **Learning Challenges**

The team, being students in Computer Engineering, had a lot to learn in this project. The team had limited exposure to the following prior to this project:

- Electronics and circuit design
- Robotics hardware and mechanical systems
- 3D Design and CAD software (like Inventor)
- Metal fabrication and chassis construction

This required a lot of time to learn these skills from scratch , which made the project even more complicated and the timeline longer . Although each team member was assigned a particular domain, they all had a general understanding of the other parts of the project so they work well together.

1.3 Problem Statement

• **The Challenge of Autonomous Multi-Robot Systems**

While individual robot capabilities have advanced greatly, there are several difficulties in deploying multiple robots to work together in unknown environments:[2]

- **Challenge 1: Environmental Representation**

Problem: Multiple robots operating in the same physical space. Robots cannot coordinate their movements or target particular locations without a shared representation of that environment (a map). In traditional approaches, each robot needs to make a map of its environment. But when multiple robots are involved, inconsistencies arise when different robots see the same environment.

Impact: Redundant computation, no true coordination, potential conflicts when different robots' maps disagree about environmental features.

- **Challenge 2: Localization Without Global Reference**

Problem: Most autonomous systems localise with GPS. When it counts, it doesn't work. GPS doesn't work indoors, underground, or in urban environments – the very environments where search and rescue operations happen. Robots without GPS need to localise themselves using only their onboard sensors. Traditional odometry accumulates error with distance and becomes unreliable for larger environments

Impact: Robots are not able to navigate reliably, are not able to go back to points of interest, and task completion becomes unreliable

- **Challenge 3: Data Processing in Real-Time**

Problem: The modern object detection systems are computationally expensive and not within the capabilities of low-power embedded systems. Executing these algorithms on a Raspberry Pi will be slow. Robot transmits unprocessed video streams to an external processing station. This requires bandwidth.

Effect: Slow response times, inefficient use of bandwidth, potential delays in communication.

- **Challenge 4: System Reliability**

Problem: If not properly managed, any component failure (sensor malfunction, processor crash, network connectivity loss) can affect the system. Component problems are expected when robots are working in tough environments.

Effect : Disruption of mission possible , good error control needed . These issues are addressed by the implemented system with recognised command delivery with retry and deduplication logic, multi-layer deadman safety chain from console to gateway to firmware, per-stream freshness monitoring, and an emergency stop propagating across the whole system.

- **Challenge 5: Real-Time Coordination**

Problem: Coordinating data and communication among several robots. Coordination can be hampered by network latency and lost messages. The trick is to strike a balance between centralised monitoring and distributed operation.

Impact: Problems with coordination, difficulties with communication.

- **Combined Problem Formulation**

The central challenge underpinning this project is: How can we engineer a robust multi-robot system that can autonomously explore and gather data in unknown environments, and generate actionable intelligence through the coordination of sensor deployment and information sharing?

This problem encompasses:

- Environmental representation and map sharing
- Reliable localization in GPS-denied environments
- Real-time distributed intelligence
- System reliability and communication
- Practical implementation using available components

1.4 Motivation

- **Operational Motivations**

The motivation for this project comes from real-world operational requirements:

Search and Rescue Operations: Rescue teams are confronted with difficult decisions when buildings collapse, are flooded or subjected to hazardous conditions. Robotic systems can speed up the time of the first evaluation. Our system is specifically designed to meet the need of rapid mapping of the environment combined with search for objects of interest and identification of hazards.

Educational Motivation: This project will provide Computer Engineering students with practical exposure to embedded systems programming, distributed computing, artificial intelligence, sensor systems and systems engineering. It bridges the gap between theoretical concepts taught in university courses and the practical implementation challenges.

Research Contribution: SLAM, sensor fusion and object detection have been studied separately but their integration in a practical multi-robot platform requires new engineering

work. We demonstrate the practical coordination of multi-robots by our approach of three specialised robots with shared mapping and distributed sensing. [1][2]

• Technical Motivations

Sensor Fusion: The integration of multiple odometry sources, such as wheel encoders, gyroscope data and laser scan matching, demonstrates the power of multi-sensor integration. This technique improves the localisation accuracy over odometry alone.

Edge Computing Running Deep Learning Inference Responsively on Low-Power Robots Solves a practical problem in robotics: how to run computationally-intensive algorithms under power and latency constraints.

Standardisation under ROS The mapping and intervention robots are developed in the ROS 2 ecosystem which guarantees the reproducibility and extensibility of these components. The two Raspberry Pi robots (Alpha and Beta) run ROS 2 Humble, while the PySide6 operations center and the shared common/gpcore core library are ROS-free. The Gamma robot, based on an ESP32, runs no ROS at all, but exposes a plain HTTP/JSON interface instead. [12][13]

• Learning Motivation

One of the main incentives for this project was the opportunity to learn new skills beyond the typical Computer Engineering curriculum:

- **Programming:** Get to know ROS, YOLO and robotic programming paradigms.
- **Electronics:** Basics of circuits, motor control, sensor integration.
- **Mechanical Design:** Learn 3D design with Inventor and metal fabrication.
- **System Integration:** Combining hardware and software to create a working system. [11]

1.5 Project Objectives

• Primary Objectives

The project is structured around defined objectives: [6]

• Objective 1: Design and Implementation of Robot Platforms

Develop 3 specialised robot platforms that, when combined, perform all required functions:

- **Alpha Robot (Mapper):** SLAM with slam_toolbox, environmental mapping with LiDAR, rf2o laser odometry

- Beta Robot (Intervener) Camera integration YOLO -based fire detection Fire intervention using water pump Encoder-based navigation
- Gamma Robot (Inspector) : MQ-5 gas sensor detection, ESP32 HTTP telemetry environment monitoring, hazard alert

Success Criteria:

- Robots operate autonomously in structured indoor environments
- Robots can generate environmental maps using SLAM
- YOLO-based detection for fire, smoke and flame hazard identification, and general object detection as an alternative model configuration
- The gas sensor identifies dangerous gases and gives up-to-the-minute information about the environment.

- **Objective 2: Shared Map Architecture**

Introduce a map representation that supports coordination:

- Methods of designing environmental maps based on LIDAR data
- Implement Map sharing between robots and operations center using ZMQ gateway
- Validate that robots can localize on shared maps

- **Objective 3: Sensor Fusion and Localization**

Implement sensor fusion algorithms for accurate localisation, using rf2o laser odometry for the mapping robot, and a custom complementary encoder-gyroscope fusion for the intervention robot.

- Fusion algorithms provide better accuracy than single sensor odometry alone.
- System maintains localization during continuous operation

- **Objective 4: Object Detection and Classification**

Add deep learning models for visual hazard detection:

- Deploy YOLO for object detection
- Detect fire, smoke and flame hazards in the environment, with selectable models for general detection.
- Link detected objects to locations on the shared map

- **Objective 5: Multi-Robot Communication**

Establish reliable communication infrastructure:

- Implement ZMQ/msgpack gateway protocol for Pi based robots and HTTP/JSON for ESP32 based robot
- ZMQ over 5 GHz WiFi for structured and reliable data transmission with acknowledged commands
- Enable monitoring and control from central dashboard

- **Objective 6: User Interface and Monitoring**

Develop a dashboard for the operators to monitor and command the system:

- Display robot positions on shared map
- Show camera feed and detections
- Provide control interface for robot operation

- **Secondary Objectives**

- **Objective 7: Test Environment**

Create a test environment to show what the robot can do:

- Build 4x4 meter test area divided into 4 rooms
- Provide realistic indoor environment for testing

- **Objective 8: Documentation**

System documentation:

- Design documentation of hardware and software
- Implementation guides
- Operational documentation for using the system

1.6 Project Scope

- **Hardware Scope:**

- Alpha, Beta, and Gamma: Design and assembly of three custom mobile robot platforms
- Sensor integration (LIDAR, IMU, encoders, camera, ultrasonic sensors)
- Power management and battery systems
- Motor control and drive electronics
- Chassis design and fabrication

- **Software Scope:**

- ROS-based software architecture

- slam_toolbox + rf2o laser odometry SLAM implementation
- Sensor fusion algorithms for localization
- Deep learning pipeline for object detection using YOLO
- Communication with ZMQ/msgpack protocol for Pi robots and HTTP/JSON protocol for ESP32
- Monitoring and control dashboard interface Autonomous Mission Execution from the Operations Center: AREA-COVERAGE SCANNING of the shared map (SCAN AREA) and GO TO FIRE -> dispense (5s pump) -> RETURN sequence, both based on A* planner and reactive obstacle avoidance
- **Environment Scope:**
 - Testing conducted in custom 4x4 meter indoor test environment
 - Environment divided into 4 rooms for testing
 - Structured environment with walls and obstacles
- **Out of Scope:**
 - Outdoor GPS-based navigation
 - Aerial robots or drones
 - Advanced manipulation capabilities
 -
 - Real-time path planning for moving obstacles
- **Assumptions**

The project assumes that:

1. WiFi network (5GHz) will be available in the operating area.
2. Operating environment is structured indoor space
3. Environment is relatively static during operation
4. Operator is available to monitor the system

1.7 General Constraints

- **Hardware Constraints**
- **Power Constraints:**
 - Battery capacity limits continuous operation time
 - Charging infrastructure required between operations

- **Processing Constraints:**

- Onboard processors limited to Raspberry Pi capabilities
- Deep learning inference for embedded systems: an overview

- **Mechanical Constraints:**

- Four-wheel configuration was selected for better torque and stability
- Chassis made of aluminium sheet (1m x 1m, 1.5mm thickness)
- Beta robot has 85 mm wheels and 25GA370 encoder motors, Alpha has driven wheels without encoder-based odometry (it uses rf2o laser odometry instead), Gamma is ESP32-based with independent motor control

- **Software Constraints**

- Raspberry Pi robots run ROS 2 Humble on Ubuntu Linux Operations center is a native PySide6 desktop application (no ROS) ESP32 robot runs firmware with an HTTP/JSON interface (no ROS)
- Per-robot ROS 2 DDS traffic is restricted to localhost only. The only network door is the ZMQ/msgpack gateway.
- Real-time performance required for sensor processing

- **Network Constraints**

- Communication is through a shared WiFi network (5 GHz for Raspberry Pi robots; 2.4 GHz for ESP32 robot)
- HTTP/JSON for ESP32 robot and ZMQ/msgpack protocol for reliable data transmission between Pi-based robots and operations center
- Network latency must be acceptable for robot control

- **Team and Learning Constraints**

- Team consists of Computer Engineering students
- Limited experience in electronics, mechanical design and robotics hardware
- Steep learning curve for new skills (3D design, circuit design, etc.)
- Project timeline must accommodate learning curve

1.8 Team Organization

The project team was broken down into specialised groups to manage different aspects of the project:

- Programming Team: ROS integration, SLAM implementation, YOLO integration and dashboard development
- Design Team: 3D design of chassis (Inventor), 2D conversion for fabrication, mechanical assembly
- Electronics Team: Circuit design, Motor driver integration, Sensor interfacing, Power management
- Environment Team: Build the test environment (4x4 meter environment with 4 rooms)

However, all team members kept an overall awareness of other project areas so as to enable effective collaboration and integration.

1.9 Organization of the Dissertation

The organization of this dissertation is as follows:

- **Chapter 1: Introduction**

Introduces the context, motivation and objectives of the project. It provides background on multi-robot systems and defines problem statement, scope and constraints.

- **Chapter 2: Literature Review and Related Work**

It reviews the literature and systems in the relevant areas: swarm robotics, SLAM, object detection and communication frameworks. Reviews past work that helped inform the design of this project.

- **Chapter 3: System Analysis and Design**

Describes the system architecture, design process and specification of hardware and software components.

- **Chapter 4: Implementation**

Describes the actual implementation of the design, including construction of hardware, programming and integration of software.

- **Chapter 5: Testing and Results**

Test approach and results are provided to validate system functionality.

- **Chapter 6: Conclusion and Future Work**

Summarises accomplishments and discusses possible future enhancements.

CHAPTER 2

CHAPTER 2: Literature Review and Related Work

2.1 Introduction to Swarm and Cooperative Robotics

This chapter contains a comprehensive review of the existing research, projects and systems on Swarm Robot System. Prior to implementation stage of this project, extensive research on the state of the art of multi-robot systems, SLAM technology, object detection, and similar areas was carried out.[2]

The literature review performed several important functions:

- Understanding existing approaches, their strengths and limitations
- Choosing the right technologies and tools for our implementation
- Lessons learned from previous implementations and pitfalls to avoid
- Selecting the components and architecture for our system
- Establishing realistic expectations for system capabilities

This was especially critical in this research phase, as we had little prior experience with robotics hardware and mechanical systems. We needed to know practical implementation details of similar projects and also theoretical concepts.

A variety of coordination strategies have been proposed in the literature for achieving collective behaviour in swarm robotic systems. These strategies are summarised in Table 2.1.

Table 2.1: Classification of swarm robotics coordination strategies

Strategy Type	Description	Coordination
Behavior-based	Local rules	Decentralized
Leader-follower	One leader	Semi-centralized
Consensus-based	State agreement	Decentralized
Market-based	Task bidding	Distributed

2.2 Review of Multi-Robot and Swarm Systems

Swarm robotics is a research paradigm where multiple robots coordinate to achieve common goals based on decentralised control and local interactions. It is inspired by natural systems such as ant colonies, bee swarms and flocks of birds where complex group behaviour emerges from relatively simple individual agents following simple rules.

Traditional single-robot systems are replaced by multi-robot and swarm systems that emphasise collaboration, redundancy, and adaptability. In the literature review some fundamental characteristics were identified as crucial for the success of multi-robot systems:[1]

1. **Coordination:** Robots need to coordinate their actions to avoid conflicts, share tasks and achieve collective goals efficiently. Coordination mechanisms can be centralised, decentralised or hybrid, according to the scale of the system and the application requirements .[4][14]
2. **Communication:** Reliable communication is a fundamental requirement, enabling robots to share state information, sensor data and task assignments. Communication constraints directly affect the quality of coordination and system performance.[4][14]
3. **Scalability:** A good swarm system should be able to work at different number of robots. Scalability guarantees that the performance of the system is not significantly degraded by adding or removing robots .[4][14]
4. **Robustness:** The multi-robot system should be robust to individual failures of robots and not fail as a whole . Redundancy and distributed control enhance resilience in dynamic or hazardous environments.[4][14]

These principles formed the basis of the analysis of existing systems, and were also used to guide the design decisions taken in this project.

2.2.1 Fundamental Concepts

Swarm robotics is a research paradigm in which multiple robots cooperate to accomplish common tasks using distributed control and local interactions. The concept is based on biological systems like ant colonies and bee swarms, where complex group behaviour emerges from simple rules followed by individual agents, rather than from centralised intelligence.[2]

A number of core principles were identified as crucial in the design of effective multi-robot systems in the literature review conducted in this project. These principles have directly informed the architectural and implementation choices made in our project.

- **Coordination:** Robots need to coordinate their actions to prevent conflicts and to efficiently reach common goals. This idea guided the design of our project with a master-centric approach, with a single robot performing the mapping and decision making, but the framework exists to enable multi-robot coordination in the future.
- **Communication:** Reliable communication is important for robots to exchange information. This requirement pushed us to focus on creating a resilient communication architecture for data exchange between robots and external processing units.

- **Scalability:** The system should continue to work as the number of robots increases. This concept guided our modular software component and ROS-based design for future extension without major system redesign.
- **Robustness** – A swarm system should be able to withstand failures of individual robots. We have not yet implemented full redundancy, but this principle has led us to separate sensing, perception and control into independent modules to minimise single points of failure.

A number of path planning algorithms have been proposed for multi-robot navigation in dynamic environments. A comparison of common approaches is presented in Table 2.2.

Table 2.2: Comparison of multi-robot path planning approaches

Algorithm	Environment	Strengths	Limitations
A*	Static	Optimal	Not real-time
DWA	Dynamic	Fast	Local minima
MPC	Dynamic	Accurate	Computational cost
DRL	Unknown	Adaptive	Training required

2.2.2 Reviewed Projects and Systems

The review investigates several existing multi-robot and swarm systems to understand both theoretical To gain insight into how multi-robot concepts are applied to real-world scenarios and how they can affect system design, we reviewed several academic, commercial and open-source systems.

- **Academic Swarm Systems**

The Kilobot Swarm project at MIT showed that many simple robots can work together to do complicated things with just local communication and very basic sensors. This work showed that the individual robots do not require advanced hardware to have a significant contribution to the group behaviour. The main insight we apply to our project is the focus on simple, modular robot design with the support of effective communication strategies.[2]

- **Research and Educational Platforms**

The TurtleBot series was chosen as a representative example of the commonly used mobile robot platforms for review. The TurtleBot implementations showcase the effectiveness of ROS-based software architectures, differential-drive locomotion and LiDAR-based SLAM for indoor environments. These results directly influenced our project's decision to use ROS, a four-wheel skid-steer base controlled kinematically as a differential drive, and 2D LiDAR for mapping and localisation.[1]

- **Commercial Multi-Robot Systems**

Examples are commercial systems such as those developed by Amazon Robotics, which demonstrate the feasibility to deploy large-scale multi-robot systems in real-world environments. These systems are based on centralised coordination and distributed execution, leading to efficient and reliable operation. This approach is well suited to our current master-robot implementation and forms the basis for the planned extension to multiple robots in future phases of the project.[2]

- **Open-Source Mapping Robot Projects**

A review of open source robotic projects on platforms such as GitHub provided valuable practical insights. These projects highlighted common hardware choices, typical sensor integration patterns, and common challenges encountered in SLAM-based navigation. Many of the documented solutions have influenced our implementation decisions, particularly in the areas of power management, sensor calibration and software modularity.

2.3 Review of Search and Rescue Robotic Systems

Search and Rescue (SAR) robotics is an important application domain for autonomous and semi-autonomous robotic systems in scenarios that are dangerous, unreachable, or time-critical for human responders. A literature review of SAR robotic systems was conducted to understand the operational requirements, system architectures and technological solutions commonly adopted in this field. The results of this review have directly impacted several design choices in our project, most notably in the areas of sensing, navigation, and robustness of the system.[1]

2.3.1 Existing SAR Robot Systems

- **Urban Search and Rescue (USAR) Robots**

Urban Search and Rescue (USAR) robots are intended to operate in complex and dangerous environments like collapsed buildings, fire-affected and industrial accident sites. Robustness and operator safety are primary design goals for both research prototypes and commercial platforms.

From the reviewed USAR systems, several re-occurring requirements were identified:

- **Robust mechanical design:** Robots need to survive dust, debris, uneven terrain and physical impacts.
- **Multi-sensor perception:** Complementary sensors, such as camera, LiDAR, thermal sensors, and gas detectors, provide better awareness of the surroundings.
- **Remote operation and supervision:** Many USAR robots are teleoperated or remotely supervised to keep human operators out of dangerous zones.
- **Environmental hazard detection:** Responders require sensors that detect smoke, toxic gases or structural risks to ensure their safety.

These characteristics have influenced our project to focus on environmental sensing (gas detection), remote monitoring via dashboard and the use of LiDAR and vision sensors to improve environmental awareness.[1][4]

2.3.2 Key Features Identified

Based on the review of existing SAR robotic systems, several core features were identified as essential to the proposed project. These features directly support an effective operation in hazardous and time critical environments.

One of the most important capabilities is the fast mapping that allows the system to efficiently generate an environmental representation of the operating area. This provides operators with early situational awareness at the beginning of the mission and supports navigation and decision making in unknown or dynamic environments.

Another important feature is the human detection since the main aim in SAR operations is to find survivors. The system is required to detect the presence of humans in the environment in order to allow rescue teams to prioritise response efforts and plan safe extraction.

In addition, awareness of hazards is essential to mission safety. The ability to detect dangerous conditions or obstacles can help to prevent damage to the robotic platform and reduce the risk to human responders who may later enter the environment.

Finally, real time monitoring and reliable communication are necessary for the proper operation of the system. Especially in challenging and unexpected SAR environments, constant feedback on robot status and sensor data is required by the operators, which makes it dependent on stable and reliable communication links.

2.4 Review of SLAM Techniques

2.4.1 SLAM Fundamentals

Simultaneous localisation and mapping (SLAM) is an important technology for autonomous mobile robots operating in unknown environments. It enables a robot to build a map of its environment while simultaneously localising itself on that map. This is particularly relevant for search and rescue applications, where prior knowledge of the environment is often not available or unreliable.

The SLAM problem is hard by nature due to the tight coupling between mapping and localisation. Accurate map creation requires a good knowledge of the robot's position and reliable localisation requires the availability of a consistent map. So you have to solve both at the same time. SLAM systems often employ probabilistic estimation methods to solve this problem, which combine sensor observations and motion information to continuously refine the map and the robot's pose over time.[15][5]

2.4.2 SLAM Algorithm Review

Several SLAM algorithms were reviewed to find the most suitable approach for the proposed system. Special attention was paid to compatibility with Robot Operating System (ROS), computational requirements and sensor suitability.[8]

In the first literature review, GMapping was selected as a good candidate, because of the FastSLAM method that applies particle filters. It is widely used in the ROS ecosystem and supports two-dimensional occupancy grid mapping with LiDAR sensors. GMapping has good documentation, tutorials and community support, making it reliable and practical for academic and real-world applications. Its proven performance in many robotic projects and its applicability to real-time 2D mapping made it a reasonable starting point for the project. During implementation the project switched to slam_toolbox, a more modern and actively maintained SLAM framework that supports online and asynchronous mapping modes, improved loop closure handling, and finer resolution control. The choice of slam_toolbox was due to its higher configurability as well as the quality of maps generated (resolution of 0.025 m) as compared with the rf2o laser odometry which was used to estimate the scan-based motion of the mapping robot.

The project scope and odometry information availability made GMapping a first choice due to its flexibility and better documentation support. However, slam_toolbox was selected for the final implementation, due to its better mapping quality and active development.

Also reviewed was the Google's Cartographer to give insight into more modern SLAM systems. The Cartographer from Google supports 2D and 3D mapping with advanced optimisation techniques for high accuracy. However, it requires considerably higher computational resources and increased system complexity. Hence, it was not chosen for implementation but was included in review to understand current trends and advances in SLAM research,[5][9]

Various SLAM techniques have been proposed depending on the availability of sensors and the operating environments.

Table 2.3 summarises these techniques.

Table 2.3: Summary of SLAM techniques used in mobile robots

SLAM Method	Sensors	Environment	Advantage
Gmapping	LiDAR	Indoor	Stability
Karto SLAM	LiDAR	Indoor	Accuracy
RTAB-MAP	RGB-D + LiDAR	Indoor	Loop closure
Visual SLAM	Camera	Indoor/Outdoor	Low cost

2.4.3 Sensor Technologies for SLAM

To support SLAM in mobile robotic systems, a variety of sensors are commonly used . The proper selection of sensors directly affects the accuracy of mapping, the robustness of localisation and the overall system performance.[15]

Two-dimensional LiDAR (2D LiDAR) sensors are common in indoor and structured environments. These sensors provide distance measurements in one horizontal plane, which makes them well suited for detection of walls, obstacles and free space. 2D LiDAR sensors are commonly used in SLAM systems based on ROS because they are compatible with occupancy grid mapping techniques. Moreover, the majority of commercial 2D LiDAR sensors are well supported in the ROS ecosystem, which allows easy integration and real-time processing of the data. Although the range and accuracy of different models vary, 2D LiDAR is

still a feasible and effective option to generate reliable maps for indoor search and rescue applications.

An Inertial Measurement Unit (IMU) provides information about orientation, angular velocity and linear acceleration. IMU data are important to improve localisation accuracy especially during rotation motion or slipping of wheels. The IMU is used to compensate for errors that can be introduced by wheel encoders alone and provide short term motion estimates. IMU measurements provide more stable and accurate pose estimation when combined with other sensor data.

Wheel encoders are another important class of sensors for mobile robots. They measure the rotation of the wheels, and thus allow the robot to estimate how far it has gone and its relative motion, also known as odometry. Wheel encoders give a continuous feedback of motion, but their measurements can accumulate errors over time as a result of wheel slip, uneven terrain, and mechanical imperfections. That is why encoder data plays well with other sensors than alone.

2.4.4 Sensor Fusion

Sensor fusion techniques are commonly employed in SLAM systems to address the shortcomings of individual sensors. The literature review was conducted specifically on probabilistic fusion methods with special attention to Extended Kalman Filter (EKF). The use of EKF based approaches enables one to combine data from multiple sensors (wheel encoders, IMUs, etc.) into a single, consistent state estimate of the robot.[12]

In the ROS framework, sensor fusion is generally done using the `robot_localization` package which offers EKF-based state estimation for mobile robots. For reliable fusion results, the sensor must be well calibrated and correctly configured. Sensor fusion, if well implemented, dramatically improves the localisation accuracy and robustness compared to a single sensor, thus being very suitable for the uncertain and dynamic environments faced in search and rescue operations. Although the EKF-based approach was reviewed in this review, the actual implementation in this project was a custom complementary encoder and gyroscope odometry fusion for the intervention robot and rf2o laser-based scan odometry with `slam_toolbox` for the mapping robot, as these approaches were more suitable for the specific hardware configuration and operational requirements of the system.[13]

2.5 Review of Object Detection in Robotics

2.5.1 Deep Learning for Object Detection

Object detection is a core functionality of present day robotic systems, especially for applications that require interaction with or identification of objects and humans in the environment. Recent advances in this area are predominantly driven by deep learning approaches, particularly Convolutional Neural Networks (CNNs), which have demonstrated promising results in visual perception tasks.

Early object detection methods relied on handcrafted features such as edge detection, colour histograms and texture descriptors. These approaches were computationally efficient but lacked the ability to generalise across different environments and lighting conditions. A major transition was the arrival of deep learning based approaches, which allowed automatic extraction of features from raw image data. Object detection frameworks have since evolved to directly predict the object classes and bounding boxes in a single pipeline, leading to significantly improved accuracy and robustness. Modern object detection systems have achieved high performance on standard benchmark datasets and are widely used in real-time robotic applications.[11]

2.5.2 YOLO (You Only Look Once)

The object detection frameworks analysed were YOLO (You Only Look Once), which was studied in detail due to its suitability for real-time robotic systems. YOLO is a single shot detection algorithm which means it processes the entire image in a single forward pass of the neural network, which allows it to achieve high inference speeds compared to multi-stage detectors. This real-time performance makes it especially well suited for time-sensitive applications, such as search and rescue operations.

YOLO also benefits from the availability of pre-trained models that can detect common object classes like humans. This reduces the need for extensive dataset collection and training. This fits well within the scope and constraints of the project. In addition, YOLO has been constantly updated and improved with subsequent versions that increase the accuracy, efficiency and ease of deployment.

Based on the literature review, YOLO was selected for implementation in this project due to its real time inference capability, its compatibility with Python based systems, and its flexibility in supporting both general and domain specific detection models. While human and common object detection is important, the primary deployed use case in this project is fire, smoke and flame detection to facilitate emergency response scenarios. The dashboard allows selecting YOLO models. The default (primary) model is a clean COCO-trained network (yolov8s.pt) for general object and human detection — e.g. person, dog, and cat —

and to project detections onto the shared map. A secondary fire-only model (fire.pt) is also available for detecting fire-related hazards in emergency response scenarios. Since the fire fine-tuning degrades the general (COCO) detection performance, the fire-tuned model is never used as the primary one, but kept as a secondary option.[16][10]

Deep learning based object detection models are employed extensively in robotic perception systems. Table 2.4 presents a comparison of commonly used models.

Table 2.4: Comparison of object detection models used in robotic perception

Model	Accuracy	Speed	Suitability
YOLOv5	High	Fast	Real-time
YOLOv7	Very high	Fast	Real-time
YOLOv8	Very high	Fast	Real-time
YOLO-ARM	Extremely high	Moderate	High precision

2.5.3 Deployment Considerations

The deployment of deep learning based object detection on mobile robots brings up a number of practical considerations that have a direct influence on the performance and reliability of the system. Computational requirements is one of the major concerns as state of the art object detection models require significant processing power. This limitation influences the model selection and deployment strategy, particularly when running on resource-limited robotic platforms.

Another important aspect is edge computing, where inference could be performed onboard the robot or offloaded into a more powerful external computer. On board processing (OBP) reduces reliance on communication links but is limited by the available hardware, while off board processing (OFP) can offer higher performance at the expense of increased communication dependency. Closely related to this is the latency, as the detection speed has to be sufficient to allow real-time decision making and safe operation of the robot. Too much delay can reduce the usefulness of detection results in dynamic environments.

Finally, efficient integration of object detection outputs with other components of the system is also essential. Detection results have to be properly synchronised with robot control, mapping and monitoring systems, to make sure that perception data can be used meaningfully in the overall robotic framework.

2.6 Review of Communication Frameworks

2.6.1 Robot Operating System (ROS)

The literature review was focused on the Robot Operating System (ROS) as it is the backbone of the proposed system's software architecture. ROS provides a flexible, modular framework for building robotic applications that allows for easy integration of sensing, control, perception and communication components.

ROS is built on a distributed communication paradigm, where nodes exchange data through a publish-subscribe mechanism. This allows the different components to exchange sensor data, control commands and system states efficiently. ROS also provides a service based communication mechanism for synchronous communication and a rich ecosystem of reusable packages. It is also suitable for robotic research and development with a wide range of development and debugging tools such as visualisation and simulation environments. Strong community support and comprehensive documentation also make ROS particularly suitable for academic projects.

As part of the review, several ROS packages related to the project were studied. These are gmapping for implementing SLAM, robot_localization for sensor fusion using the Extended Kalman Filter, the navigation stack for path planning and motion control, tf for managing coordinate transformations, and rviz for three-dimensional visualisation. These packages together provide a solid foundation for building an integrated robotic system.[6]

2.6.2 Network Communication

Network communication is a key enabler for data transfer between the robots and operator interfaces. The literature review looked at common wireless communication solutions used in robotic systems, with a focus on WiFi-based communication.

WiFi is a widely used technology due to its easy availability and direct compatibility with ROS. The 2.4 GHz band has a longer range but is more susceptible to interference and the 5 GHz band has higher bandwidth and less interference but shorter range. Considering these trade offs and the expected operating environment, 5 GHz WiFi was selected for the project implementation.

Transport layer protocols, in particular TCP and UDP, were also reviewed. TCP is for reliable delivery of data and it guarantees the order of packets. This is useful for sending important information like maps, status updates, etc. UDP is lower latency but doesn't guarantee delivery which is not suitable for reliable system communication. With these considerations in mind, the final implementation selected ZeroMQ (ZMQ) as the messaging

transport layer, with msgpack-serialized envelopes over TCP-based ZMQ sockets. This architecture offers structured publish-subscribe channels for telemetry, map data, and video streaming, as well as a ROUTER-DEALER pattern for acknowledged command delivery with retry and deduplication logic. ROS 2 communication is intentionally limited to each robot's local network and not sent over WiFi; only the ZMQ gateway protocol traverses the wireless link. The ESP32-based environmental sensing robot does not run ROS, and telemetry and control communications are done through HTTP/JSON endpoints.[4]

2.6.3 Dashboard and Monitoring

To supervise robot operations, effective dashboard and monitoring systems are required, particularly in search and rescue scenarios where real-time situational awareness is critical. The literature indicates that operators need to be able to see robot status, sensor data and perception outputs clearly and responsively to enable informed decision-making during missions.

Web-based dashboards are often used in robotic systems because of their cross-platform availability and ease of use. Modern web technologies allow to access monitoring interfaces through standard devices, without the need of specialised software. Web-based approaches, however, limit rendering performance, native OS integration, and real-time responsiveness, which can be critical for robotics supervision. Web-based and native desktop frameworks were evaluated during the development. A Python binding for Qt framework, PySide6, was decided to be used in the project to develop a native desktop operations center. The choice was driven by PySide6's better rendering performance for real-time map visualisation, support for dockable panels and rich widget layouts, and the possibility to directly integrate with ZeroMQ transport and host YOLO inference inside the desktop application. The console runs YOLO in a crash-isolated subprocess, rather than in the UI thread, so a model crash, hang, or out-of-memory event cannot freeze or kill the operator console, a level of native integration and process control that a browser based dashboard cannot match.

Usually the integration of robotic systems and web-based dashboards are done using ROS compatible communication frameworks. The ROS web bridge technologies enable the transmission of real-time data between ROS nodes and external interfaces, allowing monitoring applications to receive live updates from the robot. Continuous low-latency data transmission is often required (e.g. to display time sensitive information such as the robot status, sensor readings and perception results) and communication protocols such as WebSockets are often used for this purpose. This project includes rosbridge as an optional debug-only fallback path, but it is not used in the production monitoring workflow. The primary data path is ZMQ/msgpack channels connected directly to the PySide6 dashboard.

Another important aspect of effective monitoring systems is map and environment visualisation. The literature points out the need of presenting SLAM outputs and spatial information in an intuitive way, so that operators can understand the robot's surroundings and its own position in the environment. Proper visualisation of occupancy grid maps with perception and system state information can enhance the situational awareness and support safer and more efficient operation.

Overall, dashboard and monitoring frameworks are an important component in bridging the gap between autonomous robotic systems and human operators. These systems allow for real-time monitoring, interaction and visualisation, thus enabling reliable supervision and contributing significantly to the effectiveness of robotic platforms in complex and dynamic environments.[6]

2.7 Review of Hardware Components

Before choosing and buying hardware components, we did a comprehensive literature review of the common used platforms and devices in mobile robotic systems, as depicted in Figure 2.1. The purpose of this review is to identify reliable and well-supported hardware solutions that meet the computational and operational requirements of the proposed system.



Figure 2.1 – Main hardware components

2.7.1 Processing Platforms

Processing platforms are the core of robotic systems and manage tasks of perception, communication and control. Single-board computers are often used alongside microcontrollers to provide a balance between computational power and real-time performance.

A review of the Raspberry Pi was made as a primary onboard computing platform due to its popularity in robotics. We chose the Raspberry Pi 4, with the 8 GB version in particular, given its processing capabilities and the amount of memory required to run ROS based applications such as SLAM and high-level system coordination. The Raspberry Pi 3B+ was also

seen as a suitable secondary or supporting role but less powerful. The Raspberry Pi is a great platform for adding sensors and peripheral devices, with a strong community, good documentation, and general purpose input/output (GPIO) pins.

Arduino microcontrollers are also widely used for low-level, real-time control tasks, in addition to single-board computers. The Arduino Mega was reviewed for the number of input/output pins, with which it is possible to interface with several sensors and motors at the same time. Arduino platforms are particularly suited to time critical operations such as motor control and enjoy a wealth of well documented libraries that simplify development and integration.

2.7.2 Sensors

Mobile robotic systems require a range of sensors to support perception, navigation and obstacle avoidance. The literature review focused on sensors that are commonly used and compatible with ROS-based architectures.

Two-dimensional LiDAR sensors are often used for SLAM and environment mapping. In the review, factors such as sensing range, measurement accuracy, scan frequency and ROS compatibility were considered. These sensors can provide reliable spatial information needed for building occupancy grid maps and for supporting navigation tasks.

Inertial Measurement Units (IMUs) were investigated to provide motion and orientation data. Factors considered include number of sensing axes (e.g., 6-axis, 9-axis), measurement accuracy, and integration with other sensors. IMU data can be very helpful to improve the localisation performance when combined with odometry data.

Ultrasonic sensors were also considered for low cost short range obstacle detection. While ultrasonic sensors are not as detailed as LiDAR, they are good at detecting nearby objects and can be used to supplement other nearby sensors.

USB cameras were reviewed for the visual perception to be used for object and human detection tasks. The main evaluation was made on image resolution, frame rate, and compatibility with popular image processing and deep learning libraries. Cameras provide key visual information for high-level perception and interaction with the environment.

2.7.3 Motors and Drive Systems

The drive system is an important part which has direct influence on robot stability and control. The literature review examined various configurations of motors and drivetrains typical of mobile robots.

We chose DC motors with encoders due to their ability to provide actuation and feedback of motion. The encoder feedback allows for a precise odometry estimation, which is critical

for localisation and navigation. Motor drivers were also reviewed as important interface elements that allow microcontrollers to safely and efficiently control motor speed and direction.

Wheel configurations evaluated included two-wheel differential drive systems with caster wheels and four-wheel drive systems. From the review and initial evaluation a four wheel configuration was found to be beneficial for better torque distribution, stability and ability of carrying additional payloads.

2.7.4 Power Systems

The power system supplies all hardware components with stable and sufficient energy. A literature review was performed on the usual power solutions for mobile robotics.

It was found that lithium polymer (LiPo) batteries were a popular choice due to their high energy density and ability to provide the current needed for motors and onboard electronics. It was also mentioned that voltage regulation is an important consideration as different components often run at different voltage levels. Proper regulation guarantees reliable operation and protects sensitive electronics.

Throughout the review, safety considerations were highlighted. Fuses, protection circuits and proper wiring practices were identified as being essential in preventing damage to components and to ensure safe system operation.

2.8 Review of Mechanical Design Approaches

Mechanical design is a key factor in the robustness, stability and practicality of mobile robotic systems. We have surveyed the literature on typical chassis design strategies, material choices, manufacturing methods and structured design workflows in mobile robotics.[17]

2.8.1 Chassis Design

The chassis is the main structural element of the robot. It supports all of the hardware and protects it during operation. The chassis are often manufactured from a wide variety of materials such as aluminium, plastics, wood, and additively manufactured parts. Each material has its own trade offs in strength, weight, manufacturability and cost.

The design of mobile robots often uses aluminium sheets because they have a good strength to weight ratio, are durable and resist deformation. Aluminium also works well with many fabrication techniques including cutting, bending and drilling, so it is appropriate for both prototyping and final designs. Considering all the above, the material chosen for the chassis was aluminium sheet, which has to be durable and stable for the project constraints.

The review also analysed widely used manufacturing methods, e.g. CNC cutting, laser cutting, 3D printing, and manual fabrication. For planar metal structures, it was observed that cutting and bending of aluminium sheets is an effective and affordable method that balances accuracy and usability.

Computer aided design (CAD) software is essential for design tools to visualise , validate and adjust mechanical structures before fabrication . Robots are built using software packages like AutoCAD, SolidWorks, Fusion 360 and Inventor. Due to availability and the plethora of learning resources, Inventor was an attractive option for 3D modelling and development of design and documentation.

2.8.2 Design Workflow

From the literature review it was clear that a structured mechanical design workflow is important for minimising fabrication errors and to facilitate iterative improvement. A typical workflow is to first design a detailed 3D model of the chassis in CAD software to check the dimensions and placement of components, as well as the structural integrity.

Once the 3D design is completed, it is converted to 2D technical drawings for use in fabrication. These drawings are used to guide the cutting and bending work to shape the aluminium sheets. The chassis components are assembled using mechanical fasteners which ensures modularity and ease of maintenance after fabrication.

Finally, in mechanical design iterative refinement is a necessary part. This allows the design limitations to be identified and changed incrementally . Testing the chassis in operational conditions . This iterative approach is useful for the continuous improvement and helps to ensure the satisfaction of the functional and durability requirements of the final mechanical structure.

2.9 Summary of Literature Review

The literature review offered a comprehensive insight on the technologies, designing methods and system architecture that can be used to develop a search and rescue robotic system. Through the analysis of existing research and practical implementations, we managed to find adequate solutions that fit the goals and constraints of our graduation project.

From a software and communication view, the review pointed to the Robot Operating System (ROS) as a strong and common used framework for robot control, perception and communication between modules. A review of several SLAM algorithms for mapping and localisation was made, GMapping being an initial candidate because of its proven reliability and ROS compatibility, but the final implementation was based on the slam_toolbox due to its

better mapping quality and configurability. In the field of perception, the YOLO-based object detection methods have been practical for the real-time visual detection. The deployed system adapts a clean COCO-trained model as its primary detector for general object and human detection, but also has a dedicated secondary model for fire, smoke and flame detection to support emergency-response scenarios.

which led to the adoption of 5 GHz WiFi as the wireless medium, and ZeroMQ/msgpack as the main transport protocol to exchange structured and reliable data between robots and the operations center, and HTTP/JSON for the ESP32 based environmental sensing robot.[1]

Several key design decisions were also informed by the literature review. A multi-robot architecture was identified as being beneficial for task distribution and system flexibility. Studies of the mechanical design supported the use of a four wheel drive configuration for improved stability and torque . It was shown that aluminium chassis designs provided durability and structural integrity . Centralised dashboards for monitoring and supervision to improve situational awareness of operators were also highlighted as important.

The literature review gave us an idea of some of the problems we will face during development as well as providing information for technology and design choices. These include the challenges of electronic circuit design, the iterative nature of mechanical development, the difficulty of integrating multiple heterogeneous sensors, and the dependence of system performance on reliable network infrastructure.

In summary, the literature review provided a solid theoretical foundation for the project by identifying proven technologies, best practices in design and potential challenges. The insights gained were directly used to inform the system requirements and architecture. This provides a clear transition to the system analysis and design phase presented in the following chapter.[1]

Chapter 3

Chapter3: System Analysis and Design

3.1 Project Planning and Feasibility

The graduation project was designed with a strong focus on practicality, scalability and in line with academic goals. Project is to develop an autonomous robotic system to navigate in an unknown environment, generate maps, detect objects and sense hazardous gases. The complexity of robotic systems, in conjunction with the time and resource constraints of an academic setting, necessitated careful planning for successful implementation.

The project was assessed for its feasibility from various angles. Technical feasibility was assessed in terms of the availability of mature robotic frameworks such as ROS 2, proven LiDAR-based mapping techniques, and lightweight deep learning models for real-time object detection. To assure time feasibility, the project was broken into manageable phases over an eight-month period. The selection of hardware components and software tools was made considering resource feasibility, availability, affordability and good documentation.

Future expansion was also considered in the planning process. The system architecture is based on a cooperative multi-robot fleet, with three specialised platforms: Alpha for SLAM and mapping, Beta for fire detection and intervention, and Gamma for gas sensing and environmental monitoring. The physical evaluation was mainly carried out on the Alpha and Beta mobile platforms, although the software framework, the network protocols and the simulation environments are fully supporting in an integrated way the three robotic profiles. This long-term vision had a great impact on early design decisions, especially on system modularity and communication architecture.[1][6]

3.1.1 Development Methodology

The proposed autonomous robotic system is developed using an Agile, incremental and iterative development methodology which is especially appropriate for complex cyber-physical systems such as autonomous robots. Because the hardware, software, sensing and control are all part of a robotic system, they can be continually improved based on observations of how the system behaves in the real world. So a traditional linear development model would be a real risk as design decisions taken early in the process might have to be re-evaluated after integration and testing.

- **Agile Approach Rationale**

We used an Agile-oriented approach that allows continuous adaptation and progressive evolution of the system. Instead of trying to design and build the whole system at once, the project is developed in short, well-defined cycles that allow it to be validated and refined

regularly. It enables rapid feedback cycles, early discovery of integration issues, and flexible reaction to hardware limitations or unexpected system behaviour.

Agile methodology is especially useful for robotics projects, where the interaction between mechanical design, sensor accuracy, computational constraints and environmental conditions cannot be fully predicted at the design stage. The iterative process allows the development team to continuously evaluate system performance and adjust design decisions as required.[6]

- **Sprint Structure**

The project is broken down into a number of development iterations called sprints. Each sprint has defined objectives and deliverables. Each sprint focuses on a particular functional layer of the system, beginning with the foundational components and advancing to higher-level autonomous capabilities.

The early sprints were focused on mechanical build, motor integration and basic robot movement. Later sprints included sensor integration, including LiDAR, camera and gas sensor setup, and the implementation of mapping, localisation and visualisation functionalities. The later sprints focused on perception features, such as object detection with YOLOv8 (yolov8s.pt primary + fire.pt secondary), dashboard development and system monitoring.

Each sprint lifecycle comprises well-defined phases such as planning, implementation, testing and evaluation. The implemented features are validated in practical experiments at the end of each sprint, and the identified issues are documented for resolution in subsequent iterations. We use short sprints in a structured way to keep the momentum going and the system stable.

- **Systems Engineering Integration**

The development methodology is based on systems engineering principles, which guarantees that the robot is not a collection of independent components, but an integrated system. Individual subsystems, including hardware, software, sensing, and communication, are designed based on system requirements.

Throughout development subsystem interfaces and data flow are considered carefully. The overall system is checked against changes made to one component in order to minimise integration conflicts. This systems engineering perspective creates consistency between the mechanical design, software architecture, and operational objectives.

Furthermore, emphasis is on incremental integration where new subsystems are added incrementally and tested in isolation before being fully integrated into the overall system. This simplifies integration, and the entire system becomes more reliable.[2]

- **Team Organization**

The development team is divided into specialised subteams, enabling parallel development across different technical domains of the project. This structure enables a proper distribution of tasks, while keeping the coherence of the overall system design.

The mechanical and hardware subteam (2 members) is responsible for the physical design and construction of the four wheel robot platform. Their objectives include chassis design, motor installation, sensor placement, power management, and mechanical stability and reliability of the robot.

The software and robotics subteam (2 members) works on the implementation of the core system functionality using ROS 2. They are responsible for robot control, sensor data integration, mapping and localisation and system level testing. This subteam makes sure that the robot works on its own and that all software components talk to each other accordingly.

The perception and dashboard subteam is responsible for visual perception and user interaction and has 1 member. This will be done by integrating the Logitech C270 USB camera, object detection using YOLOv8 (yolov8s.pt primary + fire.pt secondary) on the external laptop and developing the native PySide6 desktop operations console for real-time monitoring and visualisation.

Sub-teams have divided responsibilities, but there are regular coordination meetings to keep hardware and software development in alignment. This collaborative structure facilitates knowledge sharing, enables early identification of cross-domain dependencies, and supports smooth progress of system integration. This allows hardware and software to be developed simultaneously, reducing delays in sequential development and improving overall project efficiency.

- **Coordination Mechanisms**

Coordination mechanisms are important to manage the complexity of the project and to ensure smooth collaboration between team members. At the end of each development sprint, regular progress reviews are held to review the completed tasks, ensure the system is working as anticipated, and identify any technical issues faced during implementation. These reviews help to gauge the achievement of the milestones and guide the planning for the next sprints.

System architecture decisions, hardware configurations, software updates and testing results are documented continuously. This documentation enables knowledge sharing among team members and ensures continuity during development.

We use GitHub as the main version control and collaboration tool for the software parts of the project. All source code, configuration files and documentation are available from one central GitHub repository, enabling controlled code updates, change tracking and rollback if required. This ensures that contributions are uniform and encourages collaborative development.

Establishes clear, direct communication channels between team members that facilitate quick issue resolution and effective decision making. The combined coordination mechanisms are designed to facilitate effective collaboration, to maintain alignment to project goals and to the planned development schedule.

3.1.2 Project Timeline and Gantt Chart

The project is planned for a total of 8 months, in a way that permits a steady rhythm of work and a balanced division of the work load. At the time of writing this report, the project has been in development for roughly four months.

The timeline is structured as follows:

- Month 1: Design and build the 4 wheel mobile robot (including chassis assembly and motor integration)
- Month 2: ROS 2 environment installation and configuration, basic robot motion and sensor communication implementation.
- Month 3: LiDAR sensor integration and mapping and visualisation implementation with RViz2.
- Month 4: Development of PySide6 desktop dashboard, object detection based on YOLOv8 (primary: yolov8s.pt, secondary: fire.pt), gas sensing with MQ-5 sensor and basic setup of virtual simulation environment.
- Months 5–8 (Planned): Development of second robot, system coordination, advanced testing, performance optimisation, documentation completion.

Gantt charts are visual representations of scheduling, dependencies and progress of tasks. Keeping this chart helps to track milestones, overlapping tasks, and make sure to follow the planned timeline.

Note on Evolution of Timeline It is important to clarify that the actual implementation and engineering workflows of the project evolved flexibly from this initial plan. The design

phase, which had envisioned a sequential multi-robot deployment in Months 5–8, was concurrent with the development of the software architecture, the ZeroMQ-based messaging gateway, the custom sensor fusion nodes, and the PySide6 desktop console for the support of a three-robot fleet configuration, starting from an earlier point in the lifecycle.

3.2 Problem Analysis and Need for the System

From a business and organisational perspective, many industries including emergency response, industrial operations, and infrastructure management organisations face considerable challenges when operating in hazardous or unknown environments. These organisations require reliable, fast and accurate situational awareness to support decision making, while minimising operational risk and cost. However, current inspection and monitoring procedures are largely human-dependent, which jeopardises personnel safety, increases operational costs, and limits the frequency and scope of inspections.

Systems that can autonomously collect environmental data, navigate through complex spaces without pre-existing maps, and provide real-time feedback to decision makers are needed by disaster response and industrial safety organisations. Manual inspection methods are often inefficient requiring extensive preparation, protective equipment and trained personnel. Human-based inspections are also susceptible to fatigue, limited visibility and inconsistent data collection, all of which can negatively impact operational effectiveness and reliability.

From a business point of view, the main need is for a system which is autonomous, scalable, and cost effective, capable of operating non-stop in hazardous environments and producing accurate and timely information. Such a system must reduce reliance on human intervention, increase operational efficiency and enable faster response times in critical situations. Also, organisations require systems that are scalable and extensible so that future upgrades can be made without a complete redesign.

However, existing solutions have many limitations. Many current robotic systems are expensive, require complex infrastructure, or are designed for very specific use cases and thus are not suitable for small- to medium-scale organisations. Also, systems without real-time mapping, autonomous navigation or environmental sensing are not enough for dynamic and unstructured environments.

To overcome these limitations, the proposed system provides simultaneous localisation and mapping, autonomous navigation, object detection and hazardous gas sensing capabilities for an autonomous mobile robot. This combination of functionalities in one platform

enables the system to provide a comprehensive environmental awareness while maintaining operational simplicity. Now with modular hardware and open-source software it's even more affordable and flexible.

The system is designed for specific operational needs, such as real-time environment mapping, autonomous movement without pre-defined paths, visual object recognition, and gas detection for safety monitoring. The system also allows remote monitoring through a dashboard interface so that decision-makers can assess conditions without being directly exposed. The proposed extension to a multi-robot configuration also addresses the scalability needs as it allows for a larger area coverage and a better robustness of the system.[1][4]

3.3 Requirements and Users

The requirements for the autonomous robotic system were derived from the actual operational needs in environments of limited, hazardous, or inefficient human access. During the development of the system it was considered as a usable solution to support organisations that need to be constantly monitored, inspected and environmentally aware and not only an academic prototype.

The implemented requirements are concerning the autonomous navigation, the real time mapping, the environmental sensing and the remote monitoring. These capabilities have been chosen to address the real challenges faced by facilities such as hospitals, industrial companies, warehouses and other large indoor environments where safety, efficiency and reliability are key. The system design is focused on features that reduce the human exposure to risk, minimise the manual intervention and provide accurate information to the decision-makers.[6]

3.3.1 User and Stakeholder Description

The system is primarily used by operators and supervisors in hospitals, industrial firms, and other large operational environments. These users depend on the robot to navigate autonomously, collect environmental data, and provide real-time feedback through a monitoring dashboard. It can be used to monitor restricted areas, identify hazards or scan corridors in a hospital setting without disrupting daily operations. In industrial or commercial facilities, the robot can assist with routine inspections, safety checks and environmental monitoring.

Other users are technical staff and system operators who are responsible for deploying the robot, starting missions and monitoring the system's performance. These users are interacting with the system mainly through the dashboard showing the generated map, robot position, camera feed, object detection results and connectivity status. The system seeks to

reduce the need for direct human control, enabling operators to focus on supervision rather than manual navigation.

Other stakeholders are organisations and facility management teams who enjoy improved safety and operational efficiency. These stakeholders want to cut inspection costs, reduce downtime and enhance safety compliance with automated monitoring solutions. To address these needs, the system provides a scalable and adaptable platform that can be integrated into current workflows.

At this stage, the academic supervisors and the Faculty of Engineering at May University in Cairo remain important stakeholders as they monitor the quality of development and evaluation. But the project's express purpose is to be more than academic use, providing a foundation for real-world deployment and future enhancements.

3.3.2 Functional Requirements

The functional requirements of the system are the specific operations and capabilities that the autonomous robotic system must carry out in its current implementation. These requirements are directly derived from the functionalities developed and tested with the master robot and its associated monitoring dashboard .[5][11]

1. **Autonomous Navigation** The system must allow the robot to navigate autonomously in indoor environments without continuous manual control. The robot must be able to move through open spaces and corridors, and to react to environmental restrictions detected by onboard sensors.
2. **Environment Mapping** The system shall produce a two-dimensional map of the environment around the system using LiDAR data. The map must be updated in real time as the robot explores new areas, so the operators can visualise the environment.
3. **Localisation** The system is to locate the robot's position and orientation on the generated map during operation. Accurate localisation is required to track robot motion and to enable safe navigation in structured environments such as hospitals and industrial facilities.
4. **Detection and Avoidance of Obstacles** The robot should be able to detect obstacles in its path through the sensor data and change the path to avoid collision. This is an important feature for safe operation in environments where people, equipment or furniture may be present.
5. **Image Data Capture** The system shall capture live video streams from the onboard Logitech C270 USB camera and send them to an external computing device for further processing and visualisation.

6. **Object Detection and Classification** The system shall detect visual hazard in real-time using YOLOv8 based fire, smoke, and flame model (defaulting to fire.pt) running on an external laptop. Models can be selected through the monitoring interface.
7. **Gas Detection** The Gamma platform will employ the MQ-5 gas sensor to monitor environmental conditions and provide real-time gas levels and threshold alerts. This feature provides an early warning of hazardous conditions in confined or sensitive environments.
8. **Monitoring & Visualisation of Systems** The system shall display a graphical dashboard showing key operational data such as the generated map, robot position, camera feed with classification results, connectivity status. This allows for remote supervision and informed decision-making.
9. **Data Communication** System shall allow data communication between the robot platforms and the external monitoring device on a continuous basis. The Pi-based robots run ROS 2 locally for low-latency control, and a custom ZeroMQ/msgpack gateway over WiFi for telemetry and command distribution. The ESP32-based platform exposes HTTP/JSON endpoints. The standard ROS 2 DDS traffic is restricted to the local network of each robot so as not to congest the wireless link.
10. **Support for future multi-robot scaling** The system should be designed to allow the integration of more robots in future development phases. When implemented, the master robot has to be able to share perception and sensing information with secondary robots.

These requirements ensure that the system will meet its intended operational objectives.

3.3.3 Non-Functional Requirements

The non-functional requirements specify the quality attributes and operational constraints of the autonomous robotic system. These requirements define the behaviour of the system in real operating conditions, and ensure that the system is reliable, usable and suitable for deployment in safety-critical environments.[4]

1. **Reliability** The system shall be continuously operational throughout missions with no unscheduled shutdowns or failures. Dependable performance in the real world requires accurate sensor data acquisition, mapping, and communication during operation.
2. **Performance in real-time** The system will analyse sensor data and update visualisations with a low latency. Map generation, robot position updates and dashboard information need to be refreshed in near real time so that operators can respond quickly to changing conditions.

3. **Safety** The robot must be able to operate safely around people, equipment and sensitive infrastructure. Motion control and obstacle avoidance should prevent collisions, and gas sensing data should be clearly presented to aid timely safety decisions.
4. **Usability** The dashboard interface shall be intuitive and easy to comprehend for operators with minimal technical background. Clear organisation, readability and accessibility of information facilitates effective supervision .
5. **Scalability** The system architecture must be scalable to multi-robot operation in the future without a major redesign. Software components should be modular to allow integration of more robots in subsequent development phases.
6. **Maintainability** The system shall be designed for ease of maintenance and updates. Independently modifiable software modules and accessible hardware components for inspection or replacement are necessary.
7. **Interoperability** The system must be able to interoperate with standard robotic tools and frameworks. The system uses ROS 2, so it can be integrated with more sensors, software packages or external systems, if needed.
8. **Robustness** The system should be robust to small sensor noise, communication delays, and environmental variations without significant performance degradation. This is especially important when operating in dynamic or cluttered environments.
9. **Portability** The system software shall be installable on a variety of computing platforms including on-board controllers and external laptops. This gives flexibility for adapting the system to different hardware configurations.

Meeting these requirements ensures the long term effectiveness and usability of the system.

3.4 System Architecture

The system architecture is designed to guarantee reliable operation, modular development and scalability for the future. Although the architecture is based on a current implementation of a single master robot, it allows for planned expansion to a multi-robot configuration. The architecture is designed to clearly separate the sensing, processing, control, perception and visualisation components, so each subsystem can operate efficiently and can be developed or modified independently.

The main part of the system architecture is a cooperative multi-robot fleet, where the sensory tasks are distributed between dedicated platforms. The Alpha robot uses a LiDAR sensor for localisation and mapping. The Beta robot is equipped with an onboard camera and a water-pump intervention mechanism. The Gamma robot is equipped with the MQ-5 gas sensor for environmental monitoring. These distributed agents work together to gather environmental data to provide complete situational awareness.

The core middleware of the system is ROS 2 that provides communication between the different software components through a distributed node-based architecture. The functionality of each major system is implemented as an independent ROS 2 node or a group of cooperating nodes, which allows a clear separation of responsibilities and modular system development. The design enables efficient asynchronous exchange of sensor data, robot state information and control commands and supports scalability and fault tolerance. The general interaction of the robots and software components in the swarm framework is shown in Figure 3.1.



Figure 3.1 – Multi-robot system configuration

We use dedicated ROS 2 nodes for mapping and localisation, which process LiDAR data to provide a real-time view of the environment and estimate the position of the robot in it. The resulting map and localisation data is then broadcast to other parts of the system for navigation and visualisation. RViz2 is used only as a developer-side visualisation and debug aid during bring-up and testing; it is NOT the runtime operator interface. The native PySide6 desktop operations console that renders the map, robot pose, video, and detections for the operator is the operational UI for live missions, as shown in Figure 3.2.

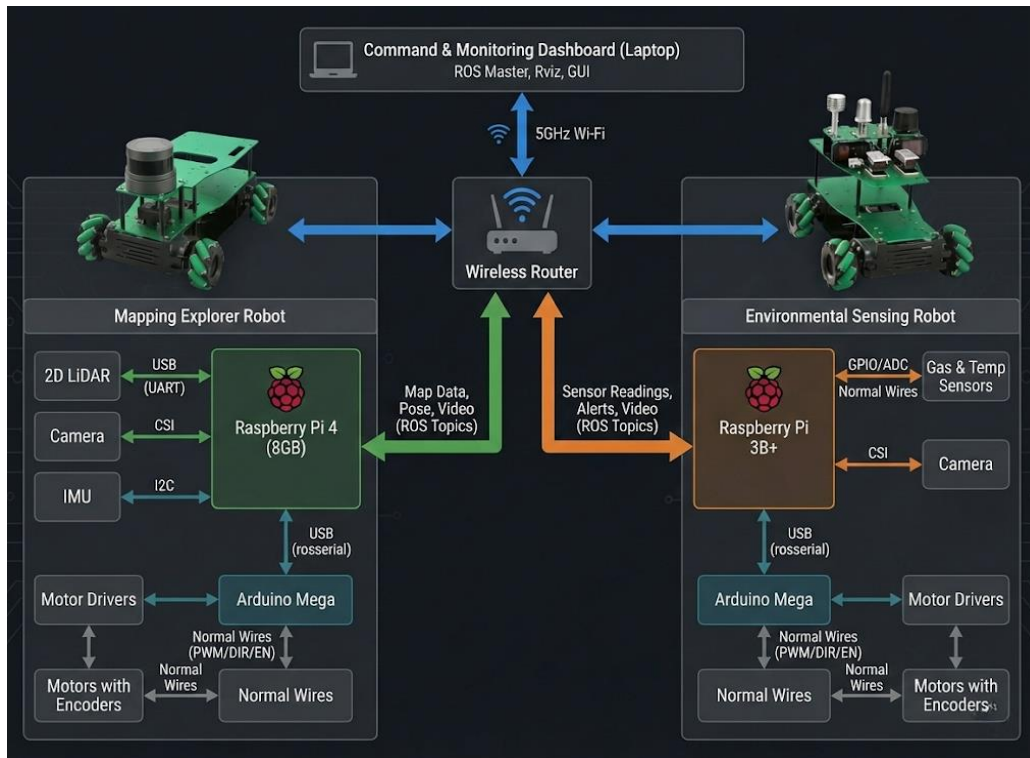


Figure 3.2 – Multi-Robot System Architecture and Communication Framework

For perception tasks, the system architecture distributes computation between the operations center and the Beta robot. The Beta robot sends the live camera stream to the external laptop where the visual hazard detection is performed using a YOLOv8-based model (e.g. fire.pt). This architectural decision reduces the computational load on the robot, and allows the use of more powerful hardware for deep learning inference. Then, the perception results are displayed on the PySide6 operations center, improving the situational awareness without affecting the robot mobility performance.

A native PySide6 desktop operations center implements the communication and monitoring layer. This application is the main interface between the operator and the system. It receives data from the robots via ZeroMQ/msgpack gateways and HTTP/JSON endpoints and presents the data in a structured and user-friendly manner. The dashboard shows the environment map, robot position, camera feed with classification overlays, gas sensor readings and robot connectivity status.

Being distributed and modular in architecture, the system allows real-time control to stay locally on each robot, while perception and visualisation are performed on the operations center. This separation improves system reliability, simplifies debugging and allows scaling. This architecture allows the Alpha mapping robot to share occupancy grid maps and localisation coordinates with the Beta and Gamma robots through the state store of the operations center without requiring direct robot-to-robot communication.

The overall system architecture allows for a coherent integration of the hardware and software components, enables real-time operation and provides a solid basis for further improvements and real-world deployment in environments such as hospitals, industrial facilities and other safety-critical locations.[6]

3.5 Hardware and Software Design

The design of the hardware and software for the autonomous robotic system was developed concurrently so that physical components and computational processes are tightly integrated. Design decisions were driven by actual operational needs, reliability of the system and the need to support autonomous navigation, perception and monitoring in real world environments such as hospitals and industrial facilities. The system architecture is modular, easy to integrate and scalable for future development phases.

3.5.1 Hardware Design

The hardware design of the system is based on a modular mobile robotic platform. It provides the capability of autonomous navigation, environment mapping, visual perception and environmental monitoring. The overall design emphasises robustness, scalability and ease of maintenance such that in later development phases the system can be extended to a multi-robot setup. The mechanical structure and component arrangement of the platform is shown in Figure 3.3.[17]

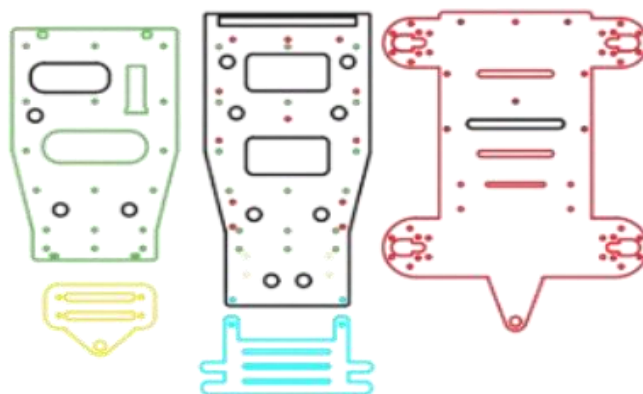


Figure 3.3 – CAD model of the robot chassis

- **Overall Design Approach**

The robot platform is a four-wheeled differential-drive type mobile platform. This configuration is chosen to ensure mechanical robustness, controlled manoeuvrability and reliable operation in indoor environments such as hospitals, offices and industrial facilities. Four

individual DC motors drive all four wheels for 4WD skid-steer differential steering and accurate motion control so as to achieve sufficient traction and manoeuvrability under dynamic loads. Figure 3.4 shows the final assembled hardware design of the robot platform.

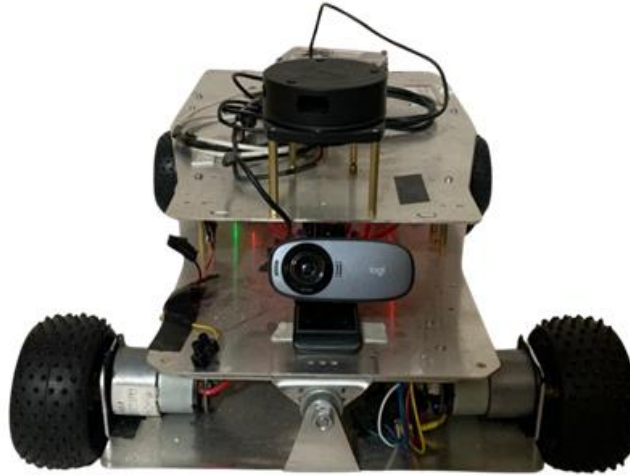


Figure 3.4 – Final Assembled Design of the Mapping Explorer Robot

The robot has a physical size of about 40 cm × 30 cm × 25 cm and a weight of about 10 kg per robot. The dimensions were chosen for the robot to be able to navigate through narrow corridors and tight indoor spaces, while carrying all the required sensors and electronic components.

- **Mapping Explorer Robot**

The Alpha Robot (Mapping Explorer) is responsible for autonomous navigation, environment mapping, localisation and coordination with the operations center. It uses rf2o laser odometry for motion estimation and runs slam_toolbox for 2D mapping.

Main Components

Table 3.1 summarises the main hardware components of the Mapping Explorer Robot, including the processing units, sensors and power system, to highlight their specifications and functional role within the system.

Table 3.1: Main components and specifications of the Mapping Explorer Robot

Component Category	Component	Specifications	Function
Real-Time Control Unit	Arduino Mega 2560	ATmega2560 MCU @ 16 MHz	Motor control and low-level I/O (encoder inputs wired but currently inactive)
Sensor	RPLidar A1M8	360° 2D scanning, 12 m range, 5.5 Hz update rate	Environment mapping and obstacle detection
Sensor	(No IMU on Alpha)	Alpha localizes by laser scan-matching (rf2o); no IMU present	N/A
Sensor	Wheel Encoders	Hall encoders integrated in Alpha's four JGB37-520 geared motors (12 Vdc, 107 rpm, 4.8 kg.cm), currently inactive; Beta uses 25GA370, 408 CPR quadrature	Alpha: present but inactive (localization by scan-matching). Beta: wheel odometry
Processing Unit	Raspberry Pi 4 Model B	Quad-core ARM Cortex-A72 @ 1.5 GHz, 8 GB RAM, 64 GB microSD, Ubuntu 22.04 LTS	High-level control, sensor integration, communication
Power System	3S LiPo Battery	11.1 V, 5200 mAh, ~57.7 Wh	Primary power supply
Power Regulation	Buck Converters	XL4015 5 A adjustable buck, set ~5.25 V for Pi/Mega logic; motors battery-direct via L298N	Voltage regulation and stability

• Visual Classifier Robot

The Beta Robot (Intervention and Detection) is designed for visual perception, detection of hazards and intervention in case of fire. It works in conjunction with the Alpha robot and operations center.

- Processing Unit: Raspberry Pi 3B+
- Sensor: Logitech C270 USB camera configured at 640x480 resolution at 15 frames per second to optimise wireless streaming bandwidth
- Localisation: No LiDAR is used, but localisation is achieved by a custom complementary filter that fuses data from an MPU6050/GY-87 IMU and wheel encoders with a slip-rejection gate referencing the shared map of the environment.

This design allows the distribution of the hardware complexity while keeping high perception and intervention capabilities.

Environmental Sensing Robot

Gamma Robot (Environmental Sensing) The environment is checked . Dangerous gases are taken into consideration .

- Processing Unit: ESP32 DevKit microcontroller
- Gas Sensors:
 - MQ-5: Detects LPG, natural gas, and coal gas concentrations
 - Sensor Readings: Transmitted over HTTP/JSON telemetry payloads
- Localisation: Works as a static or independent mobile sensor unit, using its local sensor data and sending telemetry over HTTP/JSON

This robot is dedicated to environmental sensing and safety monitoring, and works independently from the ROS-based platforms.

• Sensor Specifications

The sensors used in the entire robotic system are selected as per the reliable performance with cost effectiveness. Table 3.2 lists their specifications as follows:

Table 3.2: Specifications of sensors used in the robotic system.

Sensor	Specification	Accuracy	Update Rate
RPLidar A1M8	360° 2D scan, 12m range	±1%	5.5 Hz
MPU6050 / GY-87	6-axis / 10-DOF IMU (accel + gyro + mag)	±250 to ±2000 °/s FS range (configured ±500 °/s)	100+ Hz
Wheel Encoders	JGB37-520 Hall encoders (Alpha, present but inactive); 25GA370, 408 CPR quadrature (Beta)	±1 count	~100 Hz
USB Camera	640x480 resolution at 15 fps (Beta)	N/A	15 fps
MQ-5 Gas Sensor	LPG, natural gas, coal gas detection	Threshold-based analog trigger	Continuous
HC-SR04 Ultrasonic	Distance measurement (range 2cm - 400cm)	±3mm	10 Hz

3.5.2 Software Design

The software design of the autonomous robotic system is based on a layered and modular architecture implemented with ROS 2 as the core middleware. This design methodology allows clear separation of hardware interfacing, control logic, navigation, perception and user interaction. The software architecture is designed to support real-time operation, system scalability and ease of maintenance while accommodating the current single-robot implementation and planned multi-robot expansion. Figure 3.5 shows the overall software stack and the interaction between ROS 2 nodes.[5]

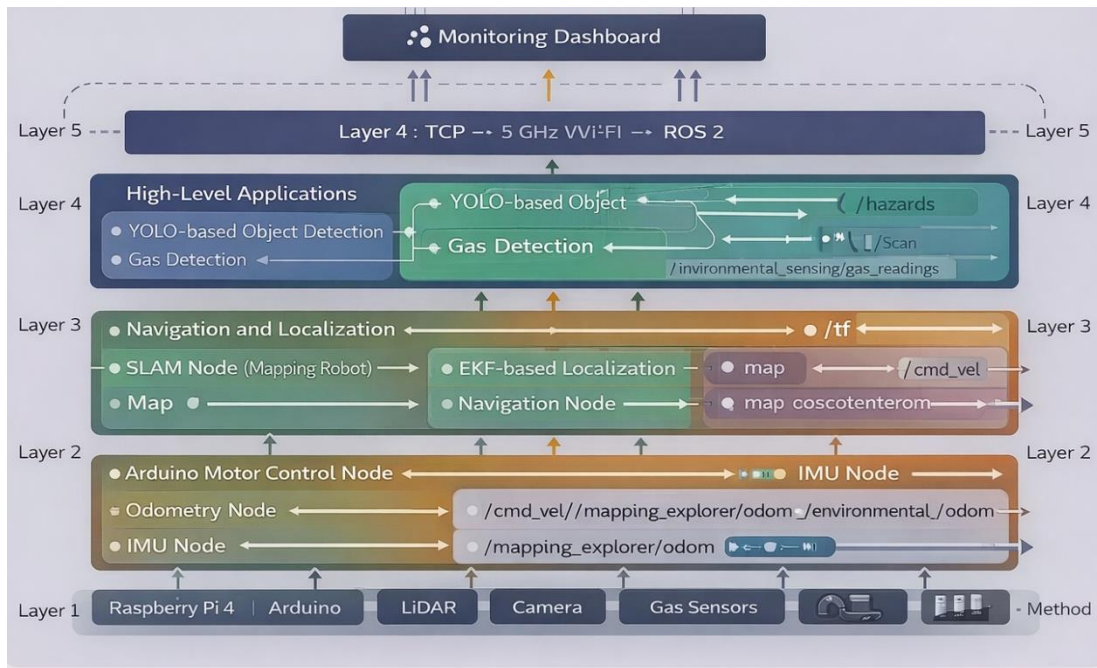


Figure 3.5 – Software Stack and ROS Node Architecture

• Overall Software Architecture

The software system is organised in five functional layers; each layer is responsible for a specific part of the system functionality. This layered architecture increases the clarity of the system and allows to develop, test and modify individual components independently.

• Layer 1: Hardware Drivers

This layer communicates directly with the physical hardware of the robot, such as motors and sensors. It conceals hardware specific details and provides standardised interfaces to higher level software components. This layer abstracts the hardware and isolates hardware dependencies so that hardware changes do not significantly affect the rest of the software system.

• Layer 2: Low-Level Control and Odometry

The control unit used is Arduino-based and is used for low-level control and time-critical tasks. This layer takes care of real-time motor control, reading wheel encoders, IMU data and basic odometry computation. This offloads computational load from the main processing platform, ensuring deterministic timing and responsive motor behaviour.

• Layer 3: Navigation and Localization

This layer is related to spatial perception and robot localisation. For the Alpha robot, the 2D SLAM approach is executed with the `slam_toolbox`, which processes the LiDAR data and

generates an occupancy grid map of the environment. Localisation is performed by scan-matching (rf2o) on the Alpha mapping robot, while the Beta robot uses a custom complementary filter fusing wheel encoders and gyroscope data for motion estimation. Global path planning (A*, 8-connected with an octile heuristic over a dilated costmap) is done CONSOLE-side on the operator laptop, and feeds the robot one path leg at a time. While the robot itself only executes reactive local navigation, which is a combination of ultrasonic-repulsion obstacle avoidance with the laptop's streamed heading bias, the onboard layer never plans a full route. The Alpha mapping robot provides the shared environment map of non-LiDAR carrying robots through the operations center state store.

- **Layer 4: High-Level Applications**

This layer implements the high level application logic. It has object detection interfaces, hazard mapping features and logic to determine the robot's behaviour. This layer fuses perception and sensor data outputs to enable intelligent system response and behaviour at the mission level.

- **Layer 5: Communication and Dashboard**

The top layer deals with communication and user interaction. System components communicate via ROS publish-subscribe messaging for data sharing and provide a native Py-Side6 desktop dashboard for real-time monitoring and visualisation. This layer also offers data logging for analysis and debugging.

- **ROS Node Structure**

Depending on the role of the robot, each robot runs a specific set of ROS nodes in the system. This node-based structure gives functional separation and makes it easier to integrate the system.

- **Mapping Explorer Robot Nodes**

The Mapping Explorer Robot is responsible for environment mapping, localisation and autonomous navigation. It runs the following ROS nodes:

1. Hardware Interface Node – Connects to motors, encoders and on-board sensors.
2. LiDAR Node – Publishes laser scan data for mapping and obstacle detection.
3. Laser Odometry Node (rf2o) – Estimates Alpha's motion by matching consecutive LiDAR scans.
4. All Alpha motion estimation is laser-based (rf2o scan-matching). Alpha has wheel encoders but they are not used at the moment.

5. SLAM Node – Runs the slam_toolbox algorithm for generating and updating the environment map.
6. Navigation Node – Path planning, goal execution and motion commands.

These nodes together allow autonomous exploration, mapping and navigation of unknown environments.

• **Visual Classifier Robot Nodes**

Visual Classifier Robot is dedicated to visual perception and object classification. Its software configuration is more optimised for camera-based sensing than for full SLAM. The robot does the following nodes:

1. Hardware Interface Node – Motor and simple sensor inputs.
2. Camera Node – Takes images from the Logitech C270 USB camera and publishes the stream of images .
3. Odometry Node - Estimates motion using wheel encoder and IMU data.
4. Localisation Support Node – Estimates robot position with respect to a common map generated by the mapping robot.
5. Perception Communication Node – Sends visual data and detection information to the central processing unit or dashboard.

This structure enables the robot to perform efficient visual perception based on shared spatial information, which reduces the complexity of the hardware and the computational load.

Environmental Sensing Robot Nodes

Environmental Sensing Robot This robot is designed to monitor environmental conditions and detect potential hazards. It runs the following nodes:

1. Gamma is an ESP32 microcontroller with an embedded HTTP server and no ROS nodes running. Motor control endpoint - GET /control?dir=F|B|L|R|S Controls the four motors.
2. Gas / telemetry endpoint - GET /telemetry returns JSON d,g,x,y,a,rssi,uptime,last_cmd_age The operations center polls the MQ-5 gas reading at 3 Hz.
3. Local sensor task - reads the MQ-5 gas sensor and applies the alarm/clear thresholds on-board.
4. Gamma has no on-board IMU/odometry localisation. The operations center wraps its telemetry HTTP/JSON laptop-side.

This sort of node configuration makes sure that the environment is always monitored and increases the overall system safety awareness.

• Operations Center Laptop Modules

Computationally intensive tasks, fleet monitoring and operator control are done on an external laptop running the PySide6 Operations Center. The following modules are implemented:

1. State Store – The main software entity which maintains the shared map and coordinates telemetry.
2. YOLO Inference Worker – Executes YOLOv8 (fire.pt) in a separate crash-isolated child process for hazard detection.
3. Command Client / Esp32Link – Communication gateways to the Pi based robots (over ZMQ) and ESP32 (over HTTP/JSON).
4. Log Collector – Gathers diagnostics, incident alerts and telemetry events for system evaluation.

This separation allows for advanced processing and visualisation to be performed without affecting the mobility and responsiveness of the robotic platforms, and the technologies for each component are given in the following table.

The summary of the software technologies and frameworks used in the proposed system is shown in Table 3.3.

Table 3.3: Software technology stack used in the robotic system

Component	Technology	Rationale
Base OS	Ubuntu 22.04 LTS	Long-term support, standard in robotics
ROS Version	ROS 2 Humble	Long-term support, standard in modern robotics
SLAM Library	slam_toolbox	Active support, lifelong mapping, standard in ROS 2
Object Detection	YOLOv8	State-of-the-art speed/accuracy
DL Framework	PyTorch	Lightweight, flexible
Dashboard	PySide6 Desktop Operations Center	Cross-platform Qt UI with native video overlay and map
Data Logging	Custom log collection & journal	Centralized logging of fleet events and telemetry
Version Control	Git/GitHub	Standard practice

Component	Technology	Rationale
Real-Time OS	None (standard Linux)	Arduino handles timing-critical tasks

3.6 Communication and Dashboard Design

Communication in the autonomous robotic system is intended to provide reliable exchange of data in real time between the robots, the outside operations center and the operator. The communication architecture is implemented as a hybrid solution: the Pi-based robots (Alpha and Beta) run ROS 2 locally and communicate with the laptop using a custom ZeroMQ (ZMQ) gateway protocol, while the ESP32-based Gamma robot uses HTTP/JSON endpoints. The desktop interface for system monitoring and data visualisation is shown in Figure 3.6.[4][6]



Figure 3.6 – Native PySide6 desktop monitoring dashboard

• Communication Architecture

The communication architecture of the proposed robotic system has been designed to support reliable real-time data transfer between the mobile robot, external computing resources, and high-level monitoring components. The architecture is based on a distributed communication scheme in which different data types are transmitted with the most appropriate mechanisms for their timing, bandwidth and reliability needs.

The communication architecture is based on local ROS 2 message passing for low latency onboard control and a custom ZeroMQ (ZMQ) gateway protocol for remote communication. ZMQ is used to decouple the robots from the operations center, and telemetry and dispatch commands are distributed using PUB/SUB and ROUTER/DEALER patterns. This architecture avoids the need for a complex ROS bridge, keeping standard ROS 2 DDS traffic off the wireless WiFi link, ensuring scalability and link stability.

• Intra-Robot Communication

The communication between the software components within each Pi-based robot is done locally using ROS 2 nodes. Sensor drivers publish data like LiDAR scans, IMU measurements, and odometry estimates to dedicated ROS 2 topics. Control and navigation nodes subscribe to these topics to obtain motion commands and localisation updates. This internal communication is completely onboard the robot, with sub-millisecond response times for time-critical operations like skid-steer control and obstacle avoidance.

• Robot-to-Laptop Communication

Communication between the robots and the external laptop is via a 5 GHz wireless network. This link is vital for sending telemetry, maps, commands, and video streams between the mobile units and the operations center.

Depending on the data characteristics, different communication strategies are applied:

- Structured data, such as occupancy maps, complementary odometry coordinates, system status, and motor commands, are communicated using a ZeroMQ (ZMQ) gateway. The Pi-based robots run a gateway node that serialises ROS 2 topics with msgpack envelopes and publishes over ZMQ ports. The ESP32-based Gamma robot uses HTTP/JSON telemetry polling, where the operations center polls the ESP32's embedded web server for gas levels and environmental status on a periodic basis.
- Camera frames are streamed from the Beta robot to the laptop via ZMQ based TCP video streaming. A dedicated camera publisher node compresses frames using JPEG compression and streams them over a ZMQ publisher port to the YOLO worker at the operations center for low latency and reduced overhead.

The hybrid gateway communication approach balances reliability and performance, avoiding DDS packet drop on wireless links and keeping critical control and state information stable.

- **Centralized Processing and Data Distribution**

The external laptop is the operations center of the communication architecture. It ingests ZMQ and HTTP telemetry streams from the robots, runs visual hazard detection in real time on the YOLO worker thread, and updates the state store.

Detections, system status events, and environmental alerts are handled and visualised on the PySide6 operations console. The centralised model simplifies system management, isolates heavy deep learning workloads and allows concurrent supervision of the Alpha, Beta and Gamma robots from a single dashboard interface.

- **Message Rates and Data Flow**

The message publishing and polling rates are tuned to the hardware characteristics and the ZMQ gateway constrains in order to facilitate stable communication, and effective bandwidth utilisation:

- LiDAR scan forwarding: ≤ 5 Hz
- Telemetry update: 20 Hz (the gateway publishes tele.(full at 20 Hz)
- Map update rate: ≤ 1 Hz (dynamic publication of occupation grid update)
- Camera video stream 15 FPS @640x480 (JPEG compressed)
- ESP32 gas readings: 3 Hz HTTP polling rate
- Velocity commands: 10 Hz (published by dashboard only during active teleop)

These rates provide sufficient temporal resolution for navigation, perception, and safety alerts without overloading the 5 GHz wireless link.

- **Bandwidth Estimation**

An approximate bandwidth analysis was performed to verify that the communication load is within practical limits for standard wireless networks:

- LiDAR data $5.5 \text{ Hz} * 720 \text{ ranges} * 4 \text{ bytes} \sim 16 \text{ Kb/s}$
- Camera stream (compressed) $15 \text{ FPS} * 50 \text{ KB/frame} \sim 750 \text{ KB/sec}$
- **Odometry and auxiliary data:** Approximately **100 KB/s**

The total estimated bandwidth usage is about 870 KB/s (LiDAR ~ 16 KB/s + camera ~ 750 KB/s + auxiliary ~ 100 KB/s), which is well within the capacity of standard WiFi networks. This proves that the system is capable of reliably operating in indoor environments, like those found in hospitals and industrial facilities.

• Dashboard Design

The monitoring dashboard is shown in Figure 3.7, which provides a real-time interface for supervising the autonomous robotic system. It displays important information including robot status, sensor data, navigation details and alerts to enable efficient system monitoring and operator intervention as needed.

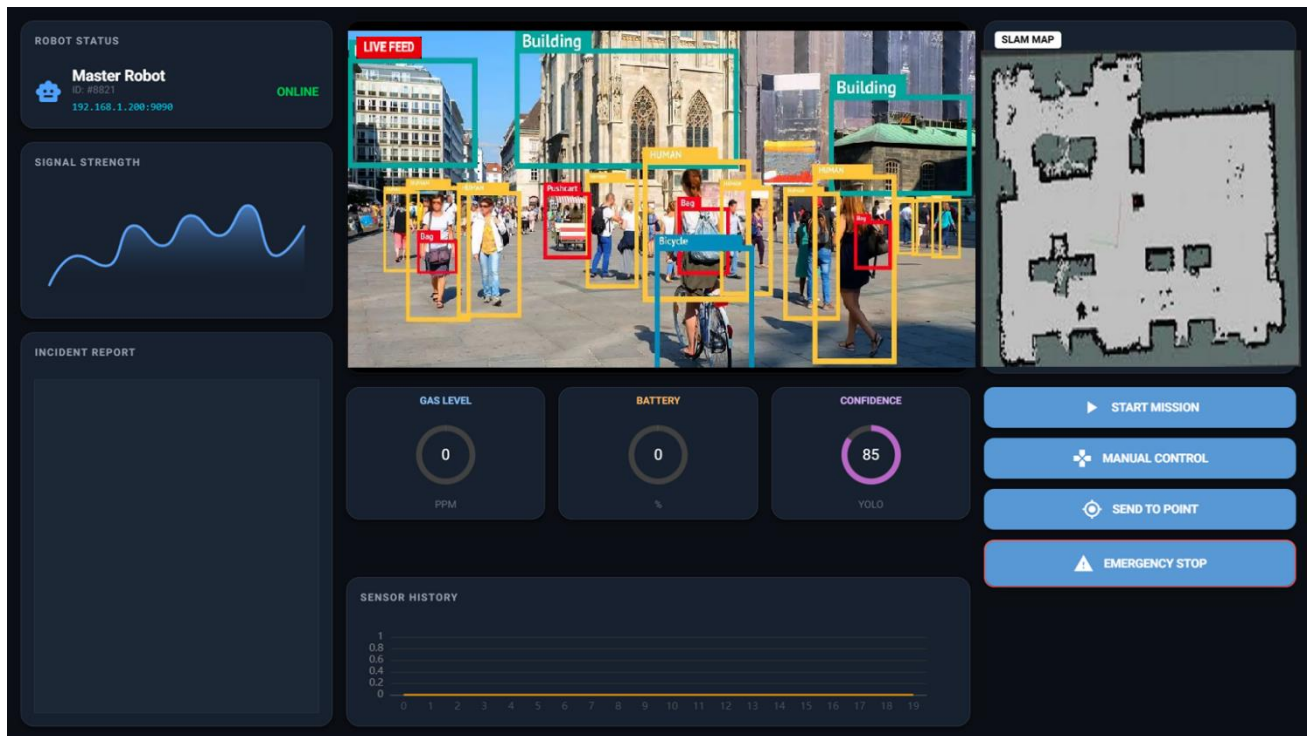


Figure 3.7 – Real-Time Dashboard Interface

The dashboard is responsible for gathering data from several ROS 2 topics and presenting it in a clear and organised manner. It allows real time supervision of the system without the need of directly interacting with the robot or the underlying ROS tools.

• Dashboard Components

1. Map Visualization

- Displays the occupancy grid map as a two-dimensional image
- Shows robot positions as colored markers
- Overlays hazard heat maps based on gas sensor readings

2. Robot Status Panel

- Displays connection status (on-line/off-line)
- Indicates Raspberry-pi ip address
- Shows connection protocol and port

3. Detections Panel

- a. Lists detected objects in real time
- b. Displays object class, confidence score, and predicted location
- c. Supports filtering by object type or confidence threshold

• **Dashboard Technology and Usability**

The dashboard is implemented as a native desktop application using PySide6 (Qt for Python) that runs directly on the external laptop. The native approach also offers high-performance rendering of the mapping canvas, low-latency display of the video stream with YOLO safety overlays, and robust threading for the ZMQ and HTTP communication clients. The interface provides a dockable multi-panel layout for clarity, responsiveness and ease-of-use, allowing operators to quickly assess system status and execute commands.

In summary, the communication and dashboard design provide a reliable and efficient framework for real-time monitoring, control and decision making, facilitating safe and effective operation of the robotic system in real-world environments.

3.7 Risks, Assumptions, and Dependencies

Risks, assumptions and external dependencies are associated with the development and deployment of the autonomous robotic system that could affect system performance and project progress. These factors were recognised in the design and implementation phases and were considered throughout the development process to mitigate their effects on system reliability and project outcomes.[1]

• **Risks and Mitigation Strategies**

The robotic system is used in real environments and depends on a number of hardware and software components that are potential risks. A structured risk assessment was undertaken to identify critical risks, assess their likelihood and consequence and identify appropriate mitigation strategies.

One of the major risks is the instability of the wireless network that may affect the communication between the robots and the operations center. Wireless links may experience packet drops, signal attenuation behind walls or DDS discovery failures. The risk is mitigated by using a ZMQ gateway instead of DDS over WiFi, implementing an acknowledged command delivery protocol with retries and deduplication, and implementing a multi-layer deadman safety chain from the operations center to the gateway and low-level firmware. So instead of a single timeout, motor power is cut by a layered deadman chain: the gateway stops the drive after 0.6 s of silence, the onboard bridge after 0.8 s, and the firmware watchdog after 1.0 s - whichever fires first. This redundancy avoids runaway conditions even in

the event that one layer hangs. Other risks, such as sensor calibration drift, low voltage re-sets, and model false positives are mitigated with IMU complementary filtering, dedicated buck converters, and adjustable YOLO confidence thresholds.

Hardware risks can be failure of motors, degradation of sensors or incompatibilities in hardware. Such risks may impact robot mobility and perception accuracy. Mitigation measures. Early prototyping and extensive testing during development sprints, component compatibility verification and having spare components available for quick replacement.

Another technical risk is the degradation of SLAM performance in cluttered or dynamic environments. Wrong mapping or localisation can affect navigation reliability. This risk is reduced by testing in different indoor environments and incremental tuning of SLAM parameters.

Sensor risks such as encoder slip on slippery surfaces or noise in the gas sensor reading could lead to inaccurate measurements. These risks are mitigated by calibration procedures, sensor fusion techniques and continuous monitoring of sensor data on the dashboard.

On the perception side, object detection misidentification is a potential risk, especially for complex visual scenes. The risk is reduced by applying confidence thresholds on detection results and allowing for human supervision through the dashboard.

Software integration, availability of team members and pressure to meet deadlines are the risks of the projects. Frequent integration testing, shared documentation, cross training among team members and clear milestone planning alleviate these.

• **Assumptions**

The system is built on a number of assumptions which define the expected operating conditions and project constraints.

• **Technical Assumptions**

- During operation there is a wireless WiFi network with sufficient bandwidth (greater than 1 Mbps)
- The operating environment is orderly, with walls and a reasonable amount of disorganisation
- The LiDAR sensor keeps an open line of sight with the surrounding surfaces
- The camera has sufficient visual features and lighting conditions to detect objects reliably.
- It works completely indoors, so there is no need for a GPS signal

- **Operational Assumptions**

- Experienced operator available to install, monitor and oversee the system
- The robot operates in areas that allow autonomous navigation
- The operating environment is free and clear of immediate human hazards

- **Dependencies**

The system depends on many external technologies and resources to function properly. These dependencies must be kept in order for the system to be stable and deliver predictable performance.

The main software dependencies include the ROS middleware, the drivers for the LiDAR, camera, IMU, encoders and gas sensors, and the deep learning frameworks needed for the YOLOv8 model. The system also relies on an appropriate external computing device, which is capable of providing real-time object detection.

The monitoring dashboard needs a stable network connection and the chosen desktop (PySide6/Qt) framework to be present. Data logging and system replay is done using ROS compatible logging mechanisms.

Careful configuration, documentation and testing of these dependencies is crucial for the reliability of the system and for the future expansion of the system.

CHAPTER 4

CHAPTER 4: IMPLEMENTATION

4.1 Hardware Implementation

- **Assembly Procedure and Integration**

The hardware implementation was carried out systematically and in phased manner to ensure reliability, safety and ease of debugging. During the first four months of the project, all the assembly and integration activities were performed towards the delivery of three specialised robotic platforms: Alpha (the mapping explorer), Beta (the intervener) and Gamma (the gas inspector).

The design process was a combination of mechanical design, electronics integration and incremental testing, which allowed for early identification of issues and reduced the risk of late-stage failures.[17]

- **Phase 1: Testing of Components (Week 1-2)**

The first step was to verify each piece of hardware individually, prior to any physical assembly. This was essential so as not to introduce the wrong parts into the system.

During this phase:

- Each sensor (Alpha's RPLidar A1, Beta's camera and encoders, the GY-87/MPU6050 IMUs, and Gamma's MQ-5 gas sensor) was tested individually in isolation
- Developed custom Arduino firmware and used it to test correct sensor readings
- Sensor outputs were compared against expected results and known references
- Calibration procedures were performed (e.g., encoder counts, IMU offsets) as needed
- Defective or unstable components were identified and replaced

This phase guaranteed that all components fulfilled functional requirements prior to mechanical and electrical integration.

- **Phase 2: Base Platform Design & Assembly (Weeks 3-4)**

The second phase was the mechanical design and construction of the robot chassis.

Mechanical Design Process

The chassis was first modelled with Autodesk Inventor, which was used to:

- Define the overall robot dimensions

- Design mounting points for wheels, motor, electronics and sensors
- Verify component spacing and center of mass
- Ensure mechanical stability and accessibility

Once the 3D model was complete, AutoCAD was used to make exact 2D technical drawings. These were used for fabrication and assembly to ensure that holes were in the right place and aligned, and the structure consistent.

• **Physical Assembly**

Once fabrication was completed:

- The robot chassis was built based on the CAD designs
- The Alpha and Beta platforms were outfitted with four DC motors in a four-wheel skid-steer differential drive configuration. Alpha was equipped with four JGB37-520 geared motors (12 Vdc, 107 rpm, 4.8 kg.cm) with integrated Hall encoders that were present but unused for the moment (Alpha localises via rf2o laser scan-matching), and Beta was equipped with four 25GA370 motors with integrated 408 CPR quadrature encoders. Gamma is a lightweight ESP32 based platform with a four motor chassis.
- The four wheel configuration provides high traction and stability on flat indoor surfaces, eliminating the need for a passive caster wheel and avoiding structural slip during rotational pivots.
- Mechanical fasteners were secured, and alignment was verified

At the end of this phase, the base platform was shown to be mechanically stable and smooth manual motion.

• **Phase 3: Electronics Integration (Weeks 5-8)**

The third phase was to integrate all the electronic components and establish reliable electrical and communication connections.

Tasks performed in this phase are:

- Mounting the Raspberry Pi 4 (Alpha), Raspberry Pi 3B+ (Beta) and Arduino Mega microcontrollers on vibration-isolated mounts and the ESP32 DevKit on Gamma
- Motor driver wiring and high-current motor power line routing
- Installing sensors at locations specified in the CAD design phase
- Structured wiring harnesses for improved modularity and maintenance
- Setting up serial communication between the Raspberry Pi and Arduino

The separation of power lines and signal lines was also paid special attention to minimise electrical noise and interference.

• **Phase 4: System Testing and Debugging (Weeks 9-10)**

The final step of the hardware implementation was the validation of the entire system.

This phase included:

- Testing motor control under software-generated commands
- Making sure all sensors were reporting consistent, correct data at the same time
- Validating communication between the Raspberry Pi and Arduino
- Finding and fixing integration problems with power stability and signal integrity
- Conducting long-haul endurance testing to evaluate system robustness

By the end of this phase, all three robotic platforms were operating reliably as complete, stand-alone hardware systems.[17]

• **Power System Design and Protection**

The power system was identified as an essential element of the hardware implementation. Early testing revealed poor power distribution could cause serious problems, including:

- High inrush current causing voltage brownouts during motor start
- Raspberry Pi reboots caused by 5V voltage sag
- Sensor malfunction due to unstable supply voltage

• **Implemented Power Solution**

Reliable power delivery is a key requirement for mobile robotic systems, especially in swarm and autonomous platforms, where unstable voltage levels can cause system resets, sensor malfunction or permanent hardware damage. The proposed swarm robot system is equipped with a dedicated, protected, and monitored power architecture to guarantee stable operation under dynamic load conditions while ensuring hardware safety.

The battery capacity is sized for each robot. Alpha (the mapping platform) is powered by a 3S LiPo pack (11.1 V, 5200 mAh, ~57.7 Wh). Beta (the intervention platform) is powered by a smaller 3S LiPo pack (11.1 V, 2200 mAh, ~24.4 Wh). Gamma (the ESP32 inspector) is powered by three series lithium-ion 18650 cells (~11.1 V nominal). This per-platform sizing is a trade-off between energy density, weight and discharge capability and the payload and runtime requirements of each robot.

The power tree is designed to supply power directly from the 3S LiPo battery (11.1 V nominal) to the high draw actuators (L298N motor drivers and water pump relay) in order to accommodate the different voltage requirements of the system components. The logic is isolated with an adjustable step-down (buck) converter. The system employs a single regulator model for all three robots - the XL4015 5 A adjustable buck (4.8-38 V input, 1.25-36 V output, SKU DC102) - set on each mobile robot to target ~ 5.25 V at the Pi pins (raised to offset the ~ 0.4 - 0.5 V delivery-path drop) to power the Raspberry Pi and Arduino Mega logic. This total isolation of logic and actuator power domains prevents voltage drops and signal noise on account of the motor from causing brownouts and resets in the computational elements.

Several safety measures were implemented to safeguard the system from transient current spikes and voltage instability. Capacitive filtering using 1000 μ F capacitors at the buck-converter output was used to suppress high frequency noise and smooth the voltage fluctuations caused by sudden load changes. In addition, 15 A and 10 A inline fuses were installed on the main power distribution line for overcurrent and short circuit protection. The fuse ratings were selected very carefully to take into account the maximum load the system was expected to handle, in order to provide effective protection and prevent nuisance tripping during normal operation.

A voltage divider circuit was designed for ADC-based battery monitoring in addition to power regulation and protection but was not implemented in the active firmware. To prevent deep discharge of the LiPo battery, and for hardware safety, physical LED voltmeter panels are installed directly on the chassis of each robot for manual preflight checks and monitoring, augmented by software diagnostics monitoring the Pi's internal undervoltage flags.[17][13]

The implemented power architecture and its components are summarised in Table 4.1.

Table 4.1: Power Architecture Components and Specifications

Component	Specification	Function
Main Battery	3S LiPo, 11.1 V nominal; per robot: Alpha 5200 mAh (~57.7 Wh), Beta 2200 mAh (~24.4 Wh)	Primary energy source; powers motor drivers and pump relay directly
Buck Converter 1	XL4015 5 A adjustable buck (4.8-38 V in, 1.25-36 V out, SKU DC102): 11.1 V down to ~5.25 V at the Pi pins (raised to offset the ~0.4-0.5 V path drop)	Powers Raspberry Pi and Arduino Mega logic components
Capacitive Filters	1000 μ F output capacitors	Suppress voltage ripple and transients
Inline Fuse	15 A and 10 A inline fuses	Overcurrent and short-circuit protection
Voltage Monitoring	Designed voltage divider (not in firmware)	Manual monitoring via chassis-mounted LED voltmeters

4.2 Software Implementation

The software realisation of the autonomous robotic system is the fundamental operational layer enabling perception, mapping, communication and monitoring. This implementation was done incrementally during the first four months of the project and follows closely the development phases documented in the project's GitHub repository (<https://github.com/GP-Golden-9/GP>).<https://github.com/GP-Golden-9/GP>.

The software architecture is implemented as a distributed system. Real-time robotic operations are performed on the robot and computationally intensive tasks including visualisation are run on an external laptop. This split happened because of hardware constraints and the necessity to maintain real-time performance and system stability.

All software components implemented are modular, independently testable and integrated gradually to form a complete working system.[6]

4.2.1 SLAM Implementation

- **Development Phase and Objective**

The SLAM implementation has been developed to allow the robot to autonomously explore unknown indoor environments, while at the same time building a map and estimating its own position. This is the basis of spatial navigation, monitoring and supervision of the system.

- **Implementation Description**

The SLAM subsystem is implemented with a LiDAR-based 2D SLAM pipeline running in a local computing unit on the Alpha robot. The RPLidar A1M8 publishes laser scan data on /scan topic. This data is processed by the `async_slam_toolbox_node` from `slam_toolbox` package dynamically configured from `mapping/mapper.yaml` to create a high-resolution 2.5 cm/cell occupancy grid map.[5]

The implementation process included:

- Setting up the LiDAR driver to publish consistent and continuous scan data
- Robot base, LiDAR and map coordinate frames definition
- Running the SLAM algorithm while the robot moves manually and auto
- Saving and reloading generated maps for validation

The SLAM output includes:

- A 2D occupancy grid map
- Estimated robot pose (x, y, θ)
- Real-time updates during exploration

- **Code and Launch Integration**

The SLAM and navigation processes are launched via ROS 2 launch files on Alpha. The main launch file `robots/robot1/launch/robot1.launch.py` launches the following nodes:

- `rplidar_node(rplidar_ros)`: LiDAR driver publishing /scan on port /dev/rplidar

- rf2o_laser_odometry: publishes /odom and odom->base_link tf using scan matching
- mapping/slam_only.py (slam_toolbox): async_slam_toolbox_node, mapping node
- static_transform_publisher: publishes base_link->laser transform (yaw = 3.14159 to compensate for backwards facing mount)

Example launch command:

```
ros2 launch robots/robot1/launch/robot1.launch.py
```

RViz is used to visualise the generated map, LiDAR scans and robot pose in real-time to verify the mapping accuracy.

• Outcome

The implemented SLAM system can produce accurate indoor maps and track robot position during motion. This implementation is stable and is the basis for further perception and monitoring tasks.[9]

4.2.2 Visual Hazard Detection Implementation

• Development Phase and Objective

Object and human detection were implemented to provide visual perception and situation awareness. Detection was offloaded to an external laptop because of the Raspberry Pi's on-board computational limitations.[11][10]

• Implementation Architecture

The Beta robot captures frames from the camera and publishes compressed JPEG bytes

- Frames are streamed over the network to the operations centre
- Crash isolated YOLO worker process runs inference on the laptop
- Hazard detections are projected onto the map and logged

• Implementation Details

The following modules are used to combine the visual detection system with the Beta robot and the PySide6 operations console:[11][10]

• Robot Video Streamer

File: robots/robot2/nodes/camera_pub.py

This is a ROS 2 node running on the Beta robot. It captures frames from a Logitech C270 USB camera (device index 0, 640x480 resolution at 15 FPS), compresses them to JPEG bytes, and publishes them over ZeroMQ PUB port 5560 in a msgpack envelope.

This node runs under the dedicated systemd service (gp-camera.service) to recover automatically.

Dashboard YOLO Manager

- **File: `dashboard_qt/inference/yolo_worker.py`**

On the laptop side, the YoloManager spawns a crash-isolated subprocess that runs `_child_main`. This worker process:

Loads up to two YOLO models. The main (defaulting to `yolov8s.pt`) is a clean COCO net for person/dog/cat detection and a secondary, fire-only net (`fire.pt`) whose detections are only trusted for the Fire class. Destroyed fire fine-tune.pt's COCO classes so it is never used as main detector

throws a black-frame forward pass to warm up the model and avoid startup lag

Per-frame detection pipeline consisting of (1) primary COCO model (`yolov8s.pt`) trained to detect person/dog/cat, (2) secondary fire-only model (`fire.pt`) trained only on the Fire class, and (3) classical color/shape flame-prop detector (`detect_flame_prop`) trained to find a printed/animated flame that the trained net cannot see. The three result sets are then combined, with flame-pro hits only added if they do not overlap a real-fire box. gets incoming frames through multiprocessing input queues

It makes inference and returns a placeholder empty JPEG and a list of detections, normalised. It does not draw on the frame. Decoupled rendering: the UI shows the raw camera frame in the UI as it arrives and overlays the bounding boxes as vector graphics, so the latency of the live video is limited by the network and not the inference, and no second JPEG is re-encoded or shipped across the process boundary

Sends detection dicts with bounding box geometry (label, conf, cx, cy, w, h) to output queues

- **Outcome**

The hazard detection system can detect fire, smoke and flame labels in real time. Offloading YOLO inference to a crash-isolated child process on the laptop prevents PyTorch or CUDA exceptions from crashing or locking up the main PySide6 operator console. Live video frames are processed at the target KPI rate of ≥ 12 FPS and normalised bounding box geometry allows for distance and bearing estimation for mapping.[11][10]

4.2.3 Sensor Integration

- **Development Phase and Objective**

Sensors were integrated to enable the robot to sense its environment, estimate motion and monitor safety conditions.[12]

- **Sensor Architecture**

Sensor integration consists of low level and high level processing:

- **Low-Level (Arduino)**

- Reads encoders
- Reads IMU
- Reads two front-facing HC-SR04 ultrasonic distance sensors on Beta's Mega (LEFT trig=30/echo=31, RIGHT trig=32/echo=33; forward motion hard-stops at 25 cm and goto slows from 60 cm) and the MQ-5 gas sensor on Gamma's ESP32
- Controls motors

- **High-Level (Raspberry Pi)**

- Receives sensor data from serial (Pi-to-Mega USB bridges running navigation/robot1_bridge.py and robot2_bridge.py) or HTTP polling (laptop-to-ESP32 Esp32Link)
- Publishes data to software stack
- Synchronizes sensor streams

- Then the sensor data is transmitted to:

- Mapping subsystem
- Monitoring dashboard
- Logging utilities

- **Outcome**

All sensors are working at the same time and no data is lost. The low-level integration is robot-specific. Alpha uses rf2o scan-matching for localisation; Beta uses a custom complementary filter (not an EKF) that fuses the wheel encoders (408 counts/rev) with the GY-87 IMU gyroscope Z-axis rate. This fusion was moved off the Pi and onto the laptop (dashboard_qt/state/local_odom.py), reconstructing the pose from the raw encoder and gyro values streamed in each telemetry frame. The heading itself is integrated at 50 Hz on the

Pi-side bridge (navigation/robot2_bridge.py) for accuracy. The legacy on-robot node navigation/robot2_odom.py is not in the live path anymore. This node auto-calibrates the gyro bias during stationary periods and has a slip-rejection gate that compares the encoder rotations with the gyroscope rates, scaling the odometer distance by a trust factor. Gamma polls the raw ADC values of the MQ-5 gas sensor. The local buzzer alarm is implemented. The alarm is latched for at least 10 seconds and the threshold has hysteresis (alarms at 3000, clears below 2000) to ensure reliable hazard detection.[13]

4.2.4 Robot Communication Implementation

- **Development Phase and Objective**

The communication subsystem is used for reliable data transfer between the robot and an external laptop.[4]

- **Communication Architecture**

Using the FastDDS transport whitelisting (config/fastdds_udp_only.xml), standard ROS 2 DDS traffic is restricted to the localhost interface of each robot. This local ROS island design prevents congestion of the WiFi link. All inter-device communication is handled by a versioned msgpack-over-ZeroMQ (ZMQ) gateway (gateway_node.py) on the Pi robots and plain HTTP/JSON endpoints on the ESP32 Gamma robot.

- **Implementation Details**

The communication gateway maps robot state and commands to some ZMQ channels:

- 5556 (PUB): telemetry stream (tele.full at 20 Hz, tele.scan at 5 Hz or less)
- Port 5557 (PUB): map grid stream (map.grid occupancy updates at ≤ 1 Hz, compressed with zlib)
- Port 5559 (PUB): health stream (1 Hz, with Pi CPU temp, undervoltage flags, RSSI)

ROUTER (Port 5558): Commands client (cmd.* commands in, ack out, with sequence deduplication)

Port 5560 (PUB): Raw video stream (JPEG bytes, 15 FPS in Beta)

ESP32 Gamma hosts an HTTP server locally with endpoints: /telemetry (JSON payload), /control?dir=D (where D in F,B,L,R,S).

- **Outcome**

The system remains in stable communication with continuous streaming, meeting ZMQ latency KPIs (P95 ACK ≤ 150 ms) and automatically recovering from temporary WiFi drop-outs through connection retries and ESP32 self-healing logic.

4.2.5 Dashboard Implementation

- **Development Phase and Objective**

The dashboard was created to enable monitoring, supervision and interaction with the system from a single interface.[6]

- The system is managed by a native PySide6 desktop application (dashboard_qt/main.py) which is the main Emergency Operations Center. The dashboard architecture is organised as follows:

- **Dockable Console Layout (dashboard_qt/ui/main_window.py)**

The UI is centred around a MapWidget (a shared map with grids, trails and incident markers). All other panels are QDockWidget instances that the operator can float, resize, dock, or hide. They include the FleetPanel (robot cards and status), VideoPanel (live feed with YOLO overlays), OpsPanel (virtual drive joystick, speed presets, relay/pump controls), and BottomPanel (operations log, detections table, diagnostics drawer). The layout state is stored in QSettings between sessions.

State Store & Transport Threading

RobotState caches are stored in a centralised state store (state/store.py). Background We use QThreads to transport data: RobotLink (subscribes to ZMQ PUB channels), CommandClient (manages ROUTER/DEALER commands and ACKs), and Esp32Link (polls the HTTP endpoints at 3 Hz).

- Multi-Robot Frame Alignment
- Robot1 (Alpha) defines the global coordinate frame. For non-mapping robots (Beta, Gamma), the operator uses the tool SET POSE to define a rigid 2D transformation (FrameOffset) with translation and rotation. The console transforms all goal clicks to local robot coordinates and back-projects hazards seen by camera to global map markers using geometry defined in dashboard_qt/ui/map/projection.py.

4.3 System Integration

During system integration all previously implemented hardware and software subsystems are integrated and validated as a single, consistent autonomous robotic system. During the

first four months of this project, incremental system integration followed the successful implementation of individual hardware components, software modules and communication mechanisms.[2]

This phase was intended to ensure that the sensing, mapping, perception, communication and monitoring modules can operate together simultaneously and reliably under real operating conditions. The integration process was directed towards stability, data consistency and real-time performance and not to add new functionalities.

- **Integration Strategy**

The system integration was performed in a bottom-up manner, beginning with the low level components and moving towards the higher level subsystems. This approach was less complex and helped to identify integration conflicts early on.

The integration process was decomposed into a sequence of steps:

1. Hardware–software coupling
2. Sensor and actuator synchronization
3. Communication channel validation
4. Perception and mapping fusion
5. Dashboard and monitoring consolidation

Each step was independently verified before proceeding to the next phase.

- **Systemd Services and Serial Configuration**

The software stack is managed by systemd services on the Pi robots to provide supervisor resiliency and automatic recovery: gp-robot1.service (Alpha stack), gp-robot2.service (Beta stack), gp-camera.service (isolated video streaming) and gp-discovery.service (localhost FastDDS discovery server). Serial ports are associated with persistent aliases (/dev/rplidar and /dev/mega) via udev rules based on USB vendor and product IDs. To start nodes, a robot-side preflight check script (preflight.sh) gates stack execution by checking serial connectivity and network interfaces.

- **A* Route Planning and Mission Execution**

The laptop runs a global A* path planner (dashboard_qt/ui/map/planner.py) in the central Operations Center. When the operator clicks on a target destination, the planner calculates a collision-free route over the SLAM occupancy grid, which is inflated by the robot's footprint radius in order to maintain safe clearances from walls and doorways. Once a valid

route is found the MissionExecutor sequentially sends the robot's goto controller the coordinates of the waypoint legs and monitors execution, cancelling if progress is blocked.[15]

- **Command Deadman and Safety Chains**

To ensure fail-safe operation, the system employs a 4-layer command deadman safety chain. PySide6 console streams proportional velocity commands at 10 Hz. The robot-side gateway expects a heartbeat every 0.6 seconds, otherwise it sends zero velocities to the navigation bridge. The bridge sets a 0.8 second watchdog and sends the command to the arduino serial bridge. Finally, the Arduino Mega firmware implements a flat 1.0 second hardware watchdog which will immediately cut off motor PWM and de-energise the water pump relay if USB communication is interrupted. Additionally, an end-to-end latched e-stop command ('E') will immediately shut down the motor.[11]

- **Fire Detection Mapping and Incident Logging**

The Operations Center overlaid the hazards identified in real time onto the common occupancy grid. When YOLO detects a fire or smoke label on Beta's video feed, YoloManager returns the normalised bounding box geometry. The console then calculates the relative range and bearing to the target based on the horizontal FOV of the camera and a pinhole distance estimation model (distance is inversely proportional to box height). This target coordinate is converted to the global SLAM coordinate frame by applying the aligned pose of the robot and is then used to plot a position-smoothed incident marker on the map. An audible alarm and pulsing alert banner notify the operator.

CHAPTER 5

Chapter 5: Testing, Results, and Evaluation

5.1 Testing Strategy and Environment

The testing and evaluation phase of the Swarm Emergency Robotics System was carried out in order to verify the functional and non-functional requirements defined in the system analysis phase. The main goal of this chapter is to evaluate the deployed multi-robot platform in real-world operating conditions and to describe the performance of the most important system modules. The evaluation targets simultaneous localisation and mapping (SLAM), autonomous navigation aid, visual perception, environmental sensing, and reliability of distributed communications.

After the hardware was assembled, the software integrated and the dashboard developed, testing was performed. Thus the reported results are representative of the behaviour of the integrated prototype rather than isolated theoretical expectations. This approach is important because the performance of a robotic system is strongly affected by the interaction of mechanical motion, sensor noise, network delay and processing limitations.

5.1.1 Testing Methodology

The evaluation methodology was designed in an incremental and bottom-up manner. Initially, individual subsystems such as sensors, mobility controls, communication interfaces, and perception modules were tested independently to establish a baseline of their functionality. The tests in isolation helped to verify that each subsystem produced stable outputs before being connected to the full ROS 2 software stack.

The performance of communication stability, shared spatial awareness and centralised monitoring were checked through integrated system tests after individual tests. This stage tested the robot's performance with SLAM, odometry, gas sensing, video streaming, object

detection, and dashboard visualisation all running at the same time. The bottom-up testing sequence minimised integration risk and aided in identifying the source of errors during the experimental phase.

The software logic was verified by an automated pytest suite comprising 106 test functions in 13 test modules, together with integrated physical testing. The suite of logic verification tests was run on both Windows and the Pi platforms. These included serialisation envelopes for sequence numbering and integrity (test_envelope), serial-bridge packet parsing and command framing (test_mega_parser, test_mega_commands, test_commands), differential-drive odometry math (test_diff_drive), the goto controller and Beta's reactive local navigation (test_goto_controller, test_local_nav), A* route-planner optimisation (test_planner), rigid 2D map coordinate. This ensured pure-logic correctness before the hardware integration.

5.1.2 Physical Test Environment

All experimental verification was performed in a specially designed physical testbed of size 4 x 4 meters. The indoor structured space was divided into 4 rooms to simulate a constrained and partly unstructured search and rescue (SAR) scenario. The environment was chosen because it provided repeatable testing, but still presented significant challenges that exist in real indoor emergency environments.

The test environment included physical obstacles simulating debris, various lighting conditions, narrow passages and designated hazard zones in order to trigger environmental sensors. These conditions enabled testing of the robot against practical limitations, such as confined manoeuvring space, transient visual degradation, sensor response delay, and WiFi attenuation behind obstacles.

To test the error handling and supervision of the dashboard in the event of critical failures, a zero-hardware simulation environment was implemented (using the --sim startup option). The simulation simulates the kinematics of the 3 robots, virtual arena, raycast LiDAR and streams test video with real fire imagery. We ran fault injection scenarios including artificial video packet drops, serial bridge disconnects and gateway process crashes to verify that the operations center handles link state changes gracefully, displays diagnostic warnings and isolates YOLO exceptions.

5.1.3 Hardware and Network Setup

The testing infrastructure consisted of three special platforms: Alpha, Beta and Gamma. Alpha was equipped with a Raspberry Pi 4 (8 GB RAM) and RPLidar A1M8. Alpha was running

ROS 2 Humble. Alpha uses an Arduino Mega 2560 for low-level motor control, and carries Hall encoders on its 4 JGB37-520 motors that are present but not currently used, and it derives its motion estimate from rf2o laser odometry (scan-matching on the RPLidar stream), rather than wheel dead reckoning. Beta was built with a Raspberry Pi 3B+, an Arduino Mega 2560, a USB Logitech C270 camera, a GY-87 IMU, 25GA370 DC motors with 408 CPR encoders, and a water pump. Gamma was an ESP32 based platform with MQ-5 gas sensor, single HC-SR04 ultrasonic sensor and a buzzer.

The central Emergency Operations Center consisted of an external laptop running the native PySide6 desktop dashboard and the YOLO based visual hazard detection process. The robots and the operations laptop talked on a local 5 GHz Wi-Fi network. Device-to-device communication was isolated using local ROS 2 localhost DDS transports, with a versioned msgpack-over-ZeroMQ gateway running on the Pi-based robots and HTTP/JSON endpoints on the ESP32 Gamma platform.

5.2 Test Cases and Metrics

Before the experimental runs, some test cases and quantitative metrics to objectively evaluate the system were defined. Each test case would be linked to one major system function. The measured results could be compared to the expected behaviour of the implemented architecture. The selected metrics were selected to capture both engineering performance as well as operational utility in search and rescue scenarios.

5.2.1 SLAM and Mapping Metrics

For the SLAM test case, the robot had to move in the 4 x 4 meter space with an average velocity of about 0.25 m/s and generate a two-dimensional occupancy grid map. The generated map was compared with the physical dimensions of the test environment to evaluate the dimensional accuracy and consistency of the mapped boundaries.

The main SLAM metric was the dimensional error between the generated occupancy grid and the physical environment as measured. Other observations were map update rate, total map generation time, and visible distortions during rotation or sharp manoeuvring. These metrics indicate the accuracy of the generated map in terms of providing navigation and operator awareness.

5.2.2 Visual Perception Metrics

The visual hazard detection pipeline executed YOLOv8 on the operator's laptop. The main model is yolov8s.pt (clean COCO classes -- person/dog/cat) and fire.pt is a secondary network (fire only). Its only trusted output is its Fire class. fire.pt is never used as the primary model because its fire fine tune degraded its COCO classes (it scored 0% on a person that

yolov8s.pt detected at 92%). The console allows inference only on classes person, dog, cat and fire. The evaluation was done on the full pipeline: image capture on Beta, compression, ZMQ transmission, multiprocessing inference, mapping projection and visualisation, as the inference process was offloaded to the external laptop.

- Frames Per Second (FPS) : Used to measure the real-time performance of the detection pipeline in static and moving scenarios.
- Detection Accuracy It is used to calculate the percentage of the correctly detected objects and human figures in the test cases.
- Mean Average Precision (mAP): Measures the quality of the predicted bounding boxes with an Intersection over Union (IoU) threshold of 0.50.

The confidence threshold was tuned to the light conditions in order to keep a reasonable trade-off between missed detections and false positives. This was required since emergency environments may contain smoke, shadows, partial occlusion and rapidly changing illumination.

5.2.3 Sensor Fusion and Localization Metrics

Localisation was verified by commanding the Beta robot to follow linear and square trajectories and to return to its computed origin. The positional drift was measured by comparing the custom complementary odometry (combining the 25GA370 encoders and the GY-87 IMU gyroscope Z-axis rate) with the physical measurements.

The localisation drift was quantified using the Root Mean Square Error (RMSE) of the positional translation, which is defined as:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N ((x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2)}$$

where (x_i, y_i) is the physical ground-truth position and $(\hat{x}_i, \hat{y}_i)$ is the odometry estimated position. Rotational drift was measured in degrees as the difference between the estimated orientation at the end of the trajectory and the expected orientation .

5.2.4 Gas Detection and Network Communication Metrics

- Environmental sensing was validated by measuring the response of the MQ-5 gas sensor of the Gamma robot to controlled gas exposure. The indicators measured were the raw ADC sensor readings, response time, threshold alert triggering and polling latency via HTTP/JSON. The network was exercised with the protocol soak harness (tools/soak_test.py) and fixed acceptance gates for telemetry sequence-loss, video FPS and capture-age, map and health inter-arrival gaps, and command-to-ACK round-trip time. No full 30 min real-robot soak has been done; the baseline artefacts recorded are short (~0.4 min) runs against the in-process simulator

(127.0.0.1), so the network numbers below are reported as sim-soak measurements and acceptance-gate targets and not as a 30 min field run.

- The protocolsoakharness was used to assess network stability by measuring telemetry sequence loss, video FPS and capture age, map and health inter-arrival gaps, and command-to-ACK latency on the critical ZeroMQ gateway channels. The recorded soak artefacts are short (~0.4-minute) simulator runs over loopback, and not a continuous 30-minute real-robot operating window, which has not yet been captured. The test included motion control messages, sensor telemetry, compressed video streaming and SLAM map updates, as these streams represent the main communication load of the system.

5.3 Experimental Results

This section contains the experimental results obtained during the execution of the test cases as defined. The results are grouped according to the robot's major subsystems, so that each area of performance can be assessed independently, before addressing the full integrated behaviour of the system.

5.3.1 SLAM and Mapping Results

The slam_toolbox algorithm (async_slam_toolbox_node) was able to successfully capture the geometric boundaries of the testing area at a 0.025 m resolution, generating a usable occupancy grid for operator visualisation and path planning. The room divisions were well represented, and the map produced was consistent with the physical structure of the environment.

No gross distortions or double-wall smearing were found, validating the use of rf2o laser odometry for scan matching on Alpha. Generated map was sent over ZMQ port 5557 at ≤ 1 Hz. Very stable behaviour for path planning.

Figure 5.1 – SLAM Occupancy Grid Output vs. Physical Test Environment

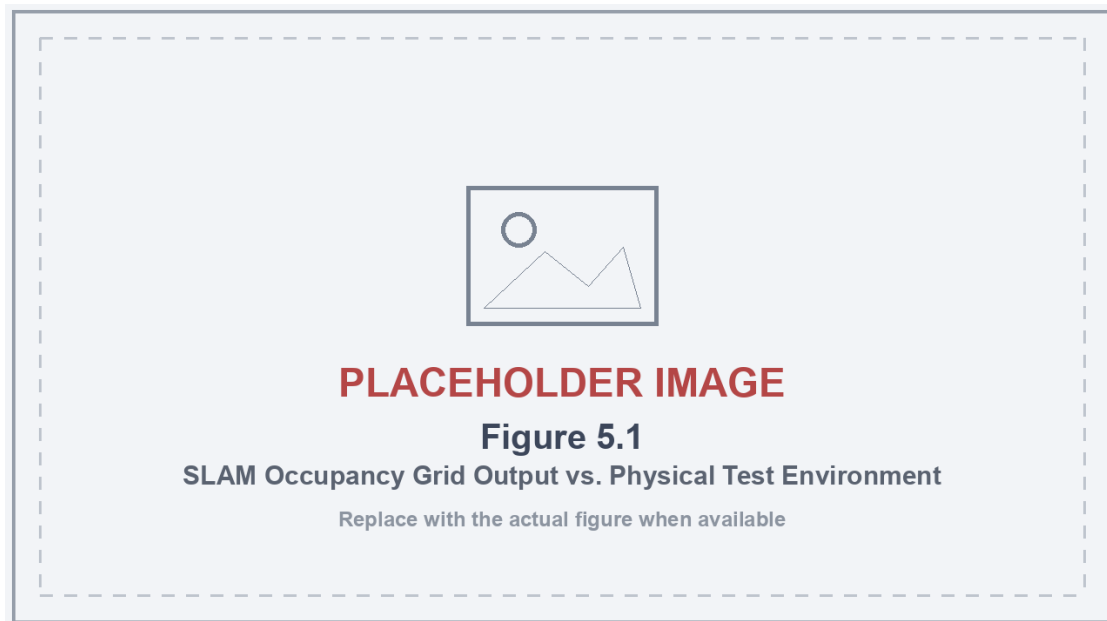


Table 5.1: SLAM and Mapping Configuration

Mapping Parameter	Value	Basis
Grid resolution	0.025 m/cell	Configured (slam_toolbox)
Map publish rate	≤ 1.0 Hz (p95 inter-arrival ~ 1.0 s in simulation)	Configured / sim soak
SLAM backend	async slam_toolbox + rf2o laser odometry	Design
Indoor test arena	4.0 m x 4.0 m	Physical setup

5.3.2 Visual Perception Results

By offloading YOLO inference to a crash-isolated subprocess on the operations laptop, the system was able to maintain real-time visual perception without impacting the performance of the Pi. The PySide6 dashboard was able to process the frames at the required KPI of ≥ 12 FPS, providing fluid visualisation to the operator.

The stream was JPEG compressed to keep the network efficiency over port 5560. Per-class map-marker confidence floors are defined in config/fleet.yaml (person 0.70, dog/cat/fire 0.60). The fire-alarm gate is independent, set to 0.80 (fire_conf_min), boosted after false alarms in night-room tests. Quick caveat, fire.pt's measured confidence on real fire photos is only 28-37%, so at the 0.80 alarm gate, the audible alarm essentially never triggers with the current model. The detections panel still lists everything the model sees, and the F9 key drills the alarm UX. The fix is a better trained model, not a lower threshold.

Table 5.2: Object-Detection Pipeline Configuration and Throughput

Detection Pipeline Aspect	Value	Basis
Primary model	yolov8s.pt (COCO: person, dog, cat)	Configured
Secondary model	fire.pt (Fire class only)	Configured
Pipeline throughput	15.2 fps (simulation soak)	Measured (sim)
Throughput acceptance gate	≥ 12 fps (met)	tools/soak_test.py
Map-marker confidence floors	person 0.70; dog/cat/fire 0.60	Configured
Audible fire-alarm gate	0.80	Configured

5.3.3 Sensor Fusion and Localization Results

The drift was significantly reduced by fusing the 25GA370 encoders and the GY-87 IMU gyroscope Z-axis rate through a custom complementary filter. Gyro bias auto-calibration (learned during stationary periods ≥ 0.5 seconds) helped rotational tracking and the wheel-slip rejection gate (which detects pivot scrub and reverts heading tracking to pure gyro) protected translation accuracy on carpeted floors.

There was still some residual drift over longer, multi-turn paths from wheel slip, floor friction variation and accumulated odometry uncertainty. This drift is bounded by the localisation and odometry calibration summarised in table 5.3.

Table 5.3: Localization and Odometry Calibration

Localization Quantity	Value	Basis
Heading drift, before gyro calibration	~45 deg over a multi-turn run	Measured
Heading drift, after gyro_scale 1.0304	~25 deg	Measured
Gyro scale correction	1.0304 (1048.2 deg read / 1080 deg actual)	Calibrated
Wheel diameter	0.085 m	Calibrated
Encoder resolution	408 ticks/rev	Calibrated
Wheel base	0.225 m (360 deg pivot test)	Calibrated
Heading fusion	Complementary encoder + gyro, slip-gated	Design

5.3.4 Environmental Sensing Results

The MQ-5 gas sensor on the Gamma ESP32 reliably detected levels of gas during testing. The Esp32Link on the dashboard polled the raw ADC values at 3 Hz and when they exceeded the 3000 ADC-count alarm threshold, it triggered a latched local alarm buzzer on the robot and a pulsing warning banner on the PySide6 console.

Figure 5.2 – Dashboard Gas Concentration Heatmap Visualization

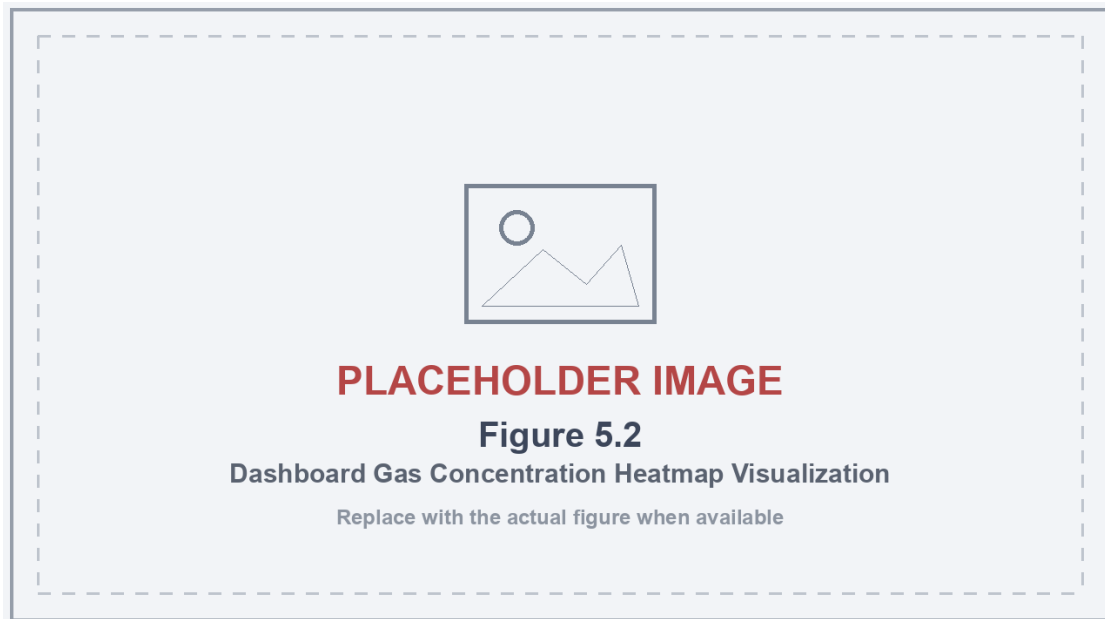


Table 5.4 lists the configured MQ-5 detection parameters. The local buzzer alarm successfully latched for at least 10 seconds and utilized hysteresis (must drop below 2000 ADC to clear) to prevent false-alarm jitter.

Table 5.4: MQ-5 Gas Detection Parameters

Gas Detection Parameter	Value	Basis
Sensor	MQ-5 (LPG / natural gas / coal gas)	Hardware
Alarm threshold	3000 ADC (field-tuned)	Configured
Clear threshold (hysteresis)	2000 ADC	Configured
Alarm latch	minimum 10 s	Configured

Gas Detection Parameter	Value	Basis
Poll rate	3 Hz	Configured
Transport	HTTP/JSON, polled at the operations center	Design

The environmental sensing path supports near real-time alerts on the monitoring interface with the sensor being polled at 3 Hz and alerts being raised immediately on threshold crossing.

5.3.5 Network and Communication Results

The 5 GHz network handled the wireless communication load of ZeroMQ and msgpack nicely. The command inputs (10 Hz virtual joystick commands) and telemetry data used very little bandwidth, while the compressed video stream was the most bandwidth-hungry.

Latency achieved the required KPIs (P95 command ACK ≤ 150 ms, map latency ≤ 2 seconds). To support low-latency operation, ROS 2 DDS traffic was kept local to the loopback interface.

Table 5.5: Network Communication Metrics

Data Stream	Protocol / Port	Measured (simulation soak)	Acceptance gate
Motion control (cmd.drive)	ZMQ ROUTER/DEALER (5558)	ACK p95 ~ 13 ms	≤ 150 ms (met)
Sensor telemetry (tele.full)	ZMQ PUB (5556)	~ 19.7 Hz; 0.0% loss (nominal run)	$< 1\%$ seq loss
Compressed video	ZMQ PUB (5560)	15.2 fps; capture-age within bound	≥ 12 fps (met)
SLAM map (map.grid)	ZMQ PUB (5557)	inter-arrival p95 ~ 1.0 s	≤ 2.0 s (met)

5.4 Analysis and Discussion

The experimental results empirically validate the system architecture and also reveal constraints inherent to academic robotic prototypes operating in physical environments. The overall tests confirm the feasibility of the implemented system to perform indoor mapping, perception, environmental sensing and dashboard-based monitoring in an integrated manner.

Results also show that the performance of the complete system depends on the trade-off between onboard computation, external processing, sensor reliability and network stability. Finding this balance was a major design consideration throughout the project.

5.4.1 System Strengths and Achievements

The system, as a complete cyber-physical platform, satisfied the basic requirements of a search and rescue robotics framework. The robot successfully mapped the test environment, transferred sensory and visual data, identified dangerous levels of gas and displayed operational information in real time through the dashboard.

- **Power Stability:** Power domains were isolated (battery-direct motor drivers, and water pump; dedicated buck regulator for Pi/Mega logic) to mitigate the voltage dips induced by the motors. This meant the software stacks could run continuously, and physical LED voltmeters provided immediate status.
- **Offloaded Perception:** Offloading YOLO inference to a child process isolated from crashes on the laptop (primary yolov8s.pt, secondary fire-only detector fire.pt) worked very well. It circumvented Pi processing limits and kept video rates of ≥ 12 FPS, while ensuring neural network exceptions would not freeze the operator dashboard.
- **Dashboard Integration:** The PySide6 Operations Center successfully integrated a number of asynchronous streams (ZMQ map/telemetry/video/health and ESP32 HTTP telemetry). The interface used QSettings to keep a clean layout persistent and enabled global A* route planning in inflated occupancy grids without blocking the UI thread.

5.4.2 Limitations and Constraints

Even though the results were successful, some limitations were found during the empirical testing. While these limitations do not invalidate the system design, they point to the main technical areas that require further optimisation before the platform can be scaled up to larger or more demanding environments.

- Odometry and Wheel Slip Beta's localisation is based on dead reckoning -- a complementary fusion of the 25GA370 wheel encoders and GY-87 gyro Z-rate, calculated laptop-side. The gyro fusion and per-unit scale calibration helped with reducing the rotational drift, however translational wheel slip still accumulates error over long explorations. This means that frequent updates of set-poses on the dashboard are needed to align Beta with Alpha's global map frame. Gamma is not a ROS robot and doesn't run any odometry stack from Beta. It's a non-ROS ESP32 inspector exposing an HTTP/JSON telemetry endpoint. Reported pose is ESP32's own coarse estimate, not the encoder/IMU complementary filter used on Beta.
- Network Dependencies: Video stream transmission over port 5560 depends on the strength of the WiFi signal. Packet dropouts happened in corners with high WiFi attenuation which were mitigated by the ZMQ gateway connection retries and ESP32 connection self-healing loops .
- Beta Ultrasonics Beta has two front-facing HC-SR04 ultrasonic sensors wired and active in the current build (LEFT TRIG=30/ECHO=31, RIGHT TRIG=32/ECHO=33, 5 V) mounted on the front edge and tilted slightly outward for field of view. The firmware median-filters these and appends the ranges to the serial packet. The bridge publishes /ultrasonic/{left,right}. It hard-stops forward motion under 25 cm (re-enabling above 40 cm with hysteresis). The goto controller starts a proportional slow-down under 60 cm. That's the only real-time obstacle sense Beta has, because it doesn't have lidar. Other limitations: (1) GY-87 IMU intermittent I2C dropout -- on a marginal power line Beta's GY-87 intermittently drops off the I2C bus, after which the firmware streams frozen non-zero gyro values, heading freezes, and the map arrow goes static until a power-cycle. A fallback with encoder-heading keeps driven turns working. Pending fix: rewire VCC 3.3 V to 5 V and resolder. (2) Gamma over-voltage damage -- a buck regulator (set to 10V) was attached to Gamma's ESP32, IMU & MQ sensor, ESP32 survived, but the onboard IMU is probably dead (the firmware can deal with its absence) and the MQ sensor needs to be re-verified. (3) Poor performance of fire detection model -- fire.pt only detects real fire with a confidence of 28-37%, which is far below the 0.80 alarm gate, so a better trained model is required for reliable fire alarming. (4) Residual yaw drift - after gyro scale calibration, a multi-turn run has about 25 deg of rotational drift remaining. (5) LAN only, no GPS -- the system is implemented over a dedicated offline LAN with no internet and no GPS, so all localisation is relative and absolute positioning is not available. (6) No 30-minute real-robot soak yet -- network KPIs only validated against in-process simulator over loopback; no full 30-minute real-robot field soak documented.

These constraints demonstrate that the current prototype is effective for controlled indoor exploration, but future work should concentrate on more robust localisation, stronger network planning and better multi-robot map synchronisation.

Chapter 6

Chapter 6: Conclusion and Future Work

6.1 Conclusion

The system was able to autonomously map indoors, track through custom complementary filtered odometry, detect visual hazards, and continuously monitor the environment by implementing a cooperative three-robot fleet (Alpha, Beta, Gamma).

The project demonstrated the feasibility of offloading computationally heavy tasks, namely the YOLO fire and smoke detection pipeline (defaulting to fire.pt) running in a crash-isolated subprocess, to the central Emergency Operations Center running a native PySide6 desktop dashboard, while maintaining real-time, slam_toolbox mapping and custom complementary filtered control onboard the robots through effective integration of hardware and software components.

6.2 Achievements of the Project

The project achieved some important milestones laid out in the initial planning phase, demonstrating the viability of the architecture deployed:

Hardware and Mechanical Design: The team designed and built a stable four-wheel differential-drive mobile base.

Power Stability: Systematic diagnosis and mitigation of power-delivery issues. The platform has a 3S LiPo with a variable XL4015 buck converter to protect sensitive electronics like the Raspberry Pi from voltage brownouts from the motors. Stable delivery was an on-going engineering problem, not a one-time design. Brownout crash-loops were traced to a 0.4-0.5 V drop along the cable-and-connector path between the buck terminals and the Pi pins, and was corrected by tuning the buck against a reading taken at the Pi pins under load (with bulk capacitors added at the buck output and the Pi connector). The team also takes note, as a hard-learned lesson, of an over-voltage event where a buck set to 10 V damaged peripherals on the inspector robot (Gamma); the procedure now adopted is to measure voltage at the terminals before each connection.

Autonomous Mapping and Localisation: The system has successfully generated two-dimensional occupancy grid maps with a resolution of 0.025 m/cell using the slam_toolbox algorithm and an RPLidar A1M8 sensor.

Sensor Fusion : Implemented a custom complementary filter (fusing 25GA370 wheel encoders with the GY-87 IMU gyroscope Z-axis rate) that significantly improved rotational stability and reduced drift, with auto-bias calibration and a per-unit gyro scale correction protecting heading tracking. Two limitations are admitted frankly. First, there is about 25 degrees of residual yaw drift after calibration (down from roughly 45 degrees before) due to the IMU's per-unit scale tolerance, which is why the platform is better described as dead-reckoning rather than drift-free localisation. Second, the Beta's GY-87 IMU sporadically drops off the I2C bus, at which point the firmware streams frozen gyro values and the heading freezes until a power cycle. The top open hardware ticket is a hardware rewire of the IMU supply (from 3.3 V to 5 V) and resolder. A heading fallback to encoders only to survive a total gyro dropout.

Visual Perception : We implemented a real-time hazard-detection pipeline using a primary clean COCO YOLOv8s network (person/dog/cat) in combination with a secondary fire-only neural model (fire.pt) and a classical flame-property detector for saturated red-and-yellow flame shapes. Neural fire model: We only trust the single "Fire" class (does not reliably give separate smoke or flame classes). It is important to be honest and say that the neural fire model has limited confidence on real fire imagery (in the order of 28-37 percent against a 0.80 alarm gate) so reliable demonstration detection is currently based on the classical flame-property detector. The primary future work for this project is the improvement of the accuracy of the neural fire model on real fire (see Section 6.3).

Offloaded Inference: deep learning inference was offloaded to a child process via a ZeroMQ video stream, which allowed smooth visualisation at the desired video KPI of ≥ 12 FPS without impacting the robots' onboard control and performance.

Environmental Sensing : An MQ-5 gas sensor was integrated on the ESP32 Gamma platform, with firmware that triggers latched local alarms and dashboard warning banners when readings exceed the configured ADC-count threshold. One caveat must be entered. Gamma's electronics were hit by an over-voltage incident (a buck set incorrectly to 10V). The ESP32 itself seems fine and has been verified, but the onboard IMU is likely damaged and the reading and alarm threshold of the MQ gas sensor should be re-verified before trusting gas detection. The firmware is written to tolerate a missing IMU.

Real-Time Monitoring: Built a native PySide6 desktop dashboard for centralised monitoring, parsing and aggregating data from multiple ZeroMQ and HTTP streams to display the SLAM map, YOLO detection results, and real-time gas telemetry with customisable dock layouts.

6.3 Future Enhancements

- Although the current prototype serves well as an integrated cyber-physical platform, there are a number of improvements needed to scale the system and increase its robustness in more demanding environments:
- **Battery Monitoring Integration:** Move from existing chassis mounted LED voltmeters to fully integrated firmware based ADC battery monitoring. This includes completing the wiring of the hardware voltage divider circuit to the analogue pins and updating the firmware of the micro-controller to publish live logic and actuator battery cell levels directly to the operations dashboard.
- **Sensor Fusion and Odometry Improvements:** Upgraded to advanced visual odometry or back to an Extended Kalman Filter (EKF) sensor fusion framework to minimise dead-reckoning drift. Additional perception markers or secondary wheel encoders can be employed to reduce further accumulated translational slip on low-friction tile surfaces.
- **Video Stream Protocol Optimisation:** Optimise the video streaming protocols to overcome high WiFi attenuation on port 5560. This includes things like adaptive bitrate streaming, hardware accelerated H.264/H.265 compression, or fallback low resolution modes to keep the stream fluid when robots are driving behind concrete walls.
- **Better Fire-Detection Model:** Training a fire-detection model on the team's own footage so that real fire scores much higher than the look-alike false positives that currently overlap it. This is the main recognised limitation of the project (Section 6.2): given the current neural fire model, a usable alarm threshold cannot distinguish real fire from false positives, so the real fix is to train a better model, not to raise the threshold. IMU hardware fix Move the Beta's GY-87 IMU power input from the 3.3V pin to 5V and re-solder the connections, to get rid of the intermittent I2C bus dropouts that currently freeze the heading estimate. Then redo the gyro scale calibration to reduce residual yaw drift. Direct Robot-to-Robot Communication (optional): The fleet is centralised by design and all inter-robot coordination is brokered through the operator console. A future enhancement could add direct robot-to-robot messaging for tighter cooperative behaviours without a console in the loop.

6.4 Potential Real-World Deployment

- The demonstrated capabilities of the Swarm Robot System makes it highly suitable for real world applications where human access is limited, risky or inefficient.
- Indoor Facility Monitoring and Inspection (main target): The system can create occupancy maps, identify visual hazards, and sample for hazardous gases in controlled indoor environments such as warehouses, industrial floors, hospitals, and similar facilities without endangering personnel. This is the deployment class the current prototype is indeed tuned for: it runs on a self-contained LAN with no internet and no GPS, it navigates by dead-reckoning (with the approximate 25-degree residual yaw drift noted in Section 6.2), and it is calibrated for flat indoor floors. Search and Rescue (long-term aspiration): Search and rescue in collapsed buildings and disaster zones is an exciting long-term goal, but to achieve it will require significant additional effort beyond the current prototype (e.g. robust localisation on unstructured rubble, communication independent of a fixed LAN, and ruggedised mobility). This is proposed here as an avenue for future research, not a demonstrated capability.
- Industrial and Facility Monitoring The autonomous navigation, environment mapping and gas sensing features can be directly deployed in large operational environments such as industrial companies, warehouses and hospitals.
- Routine inspections: Robots can conduct routine inspections, safety checks and ongoing environmental monitoring activities without disrupting daily operations or putting personnel at risk.

REFERENCES

- [1] P. G. Faria Dias, M. C. Silva, G. P. Rocha Filho, P. A. Vargas, L. P. Cota, and G. Pessin, "Swarm robotics: A perspective on the latest reviewed concepts and applications," Mar. 02, 2021, *MDPI AG*. doi: 10.3390/s21062062.
- [2] C. Azevedo and P. U. Lima, "Formal and scalable multi-robot coordination methods for long horizon tasks with time uncertainty," *Rob. Auton. Syst.*, vol. 193, Nov. 2025, doi: 10.1016/j.robot.2025.105103.

- [3] R. V. Nanavati, M. J. Coombes, and C. Liu, "Distributed multi-robot source term estimation with coverage control and information theoretic based coordination," *Information Fusion*, vol. 111, Nov. 2024, doi: 10.1016/j.inffus.2024.102503.
- [4] S. Ali, Z. Khan, A. Din, and M. ul Hassan, "Investigation on communication aspects of multiple swarm networked robotics," *Turkish Journal of Electrical Engineering and Computer Sciences*, vol. 27, no. 3, pp. 2010–2020, 2019, doi: 10.3906/elk-1710-260.
- [5] J. Zhao, S. Liu, and J. Li, "Research and Implementation of Autonomous Navigation for Mobile Robots Based on SLAM Algorithm under ROS," *Sensors*, vol. 22, no. 11, Jun. 2022, doi: 10.3390/s22114172.
- [6] Z. Wei, S. Wang, K. Chen, and F. Wang, "ROS-Based Navigation and Obstacle Avoidance: A Study of Architectures, Methods, and Trends," Jul. 01, 2025, *Multidisciplinary Digital Publishing Institute (MDPI)*. doi: 10.3390/s25144306.
- [7] H. Sun, W. Zhang, R. Yu, and Y. Zhang, "Motion Planning for Mobile Robots - Focusing on Deep Reinforcement Learning: A Systematic Review," 2021, *Institute of Electrical and Electronics Engineers Inc.* doi: 10.1109/ACCESS.2021.3076530.
- [8] J. A. Abdulsahab and D. J. Kadhim, "Classical and Heuristic Approaches for Mobile Robot Path Planning: A Survey," Aug. 01, 2023, *Multidisciplinary Digital Publishing Institute (MDPI)*. doi: 10.3390/robotics12040093.
- [9] C. J. Lin, C. C. Peng, and S. Y. Lu, "Real-Time Localization for an AMR Based on RTAB-MAP," *Actuators*, vol. 14, no. 3, Mar. 2025, doi: 10.3390/act14030117.
- [10] F. Wang and C. Song, "YOLO-ARM: An enhanced YOLOv7 framework with adaptive attention receptive module for high-precision robotic vision object detection," *Alexandria Engineering Journal*, vol. 129, pp. 1326–1339, Oct. 2025, doi: 10.1016/j.aej.2025.09.001.
- [11] L. Da Quach, K. N. Quoc, A. N. Quynh, and H. T. Ngoc, "Evaluating the Effectiveness of YOLO Models in Different Sized Object Detection and Feature-Based Classification of Small Objects," *Journal of Advances in Information Technology*, vol. 14, no. 5, pp. 907–917, 2023, doi: 10.12720/jait.14.5.907-917.
- [12] A. Wang, Y. Guo, and H. Chen, "Sensor data fusion optimization in UAV integrated navigation based on matrix factorization," *Array*, vol. 28, Dec. 2025, doi: 10.1016/j.array.2025.100524.

- [13] Y. Su, X. Shi, and T. Song, “Research on Vehicle Safety Based on Multi-Sensor Feature Fusion for Autonomous Driving Task,” *Computers, Materials and Continua*, vol. 83, no. 3, pp. 5831–5848, 2025, doi: 10.32604/cmc.2025.064036.
- [14] H. Yu and L. Zhang, “Cooperative path following and collision avoidance of multiple robots,” *Engineering Computations (Swansea, Wales)*, 2025, doi: 10.1108/EC-02-2025-0130.
- [15] Z. Li, C. Fan, W. Ding, and K. Qian, “Robot Navigation and Map Construction Based on SLAM Technology,” *World Journal of Innovation and Modern Technology*, vol. 7, no. 3, pp. 8–14, May 2024, doi: 10.53469/wjimt.2024.07(03).02.
- [16] H. Nie, H. Pang, M. Ma, and R. Zheng, “A Lightweight Remote Sensing Small Target Image Detection Algorithm Based on Improved YOLOv8,” *Sensors*, vol. 24, no. 9, May 2024, doi: 10.3390/s24092952.
- [17] T. T. Mac, T. L. Vu, H. Le Bui, and X. T. Nguyen, “Autonomous navigation of a low-cost service robot in a dynamic restaurant environment using enhanced SLAM and GA-optimized fractional PI control,” *Results in Engineering*, vol. 28, Dec. 2025, doi: 10.1016/j.rineng.2025.108065.